ChargeSaathi (EV Charging Station Locator)

A PROJECT REPORT

Mini Project (01CE0609)

Submitted by

Devicharan Dasari
92410103020

Rudra Thakker
92410103026

BACHELOR OF TECHNOLOGY

in

Computer Engineering

Faculty of Engineering & Technology

Marwadi University, Rajkot

April, 2026

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled ChargeSaathi (EV Charging Station Locator) has been carried out by Devicharan Dasari 92410103020, Rudra Thakker 92410103026 under my guidance in partial fulfillment for the degree of Bachelor of Technology in Computer Engineering, 6th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof Viraj Daxini	Prof. (Dr.) Krunal Vaghela
Assistant Professor	Professor & Associate Dean
Department of Computer Engineering	Department of Computer Engineering

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

DECLARATION

We hereby declare that the **Mini Project (01CE0609)** report submitted along with the Project entitled ChargeSaathi (EV Charging Station Locator) submitted in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering to Marwadi University, Rajkot, is a bonafide record of original project work carried out by me / us at Marwadi University under the supervision of **Prof Viraj Daxini** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

	Name of the Student	Sign of Student
1	_____	_____
2	_____	_____

Acknowledgement

It is our honour and privilege to thank those people who have helped in the successful development of this Mini Project titled "ChargeSaathi (EV Charging Station Locator)."

To begin with, we would like to thank our internal guide Prof. Viraj Daxini for supporting us throughout this project. The constant help and guidance provided by them has made it possible for us to design this full stack application.

Moreover, we would also like to thank the other professors and lab assistants and the Department of Computer Engineering/IT at Marwadi University who supported us indirectly in overcoming technical issues.

The combined efforts made by both of us towards its development have enabled us to develop this Mini Project.

Abstract

ChargeSaathi appears as a smooth, up-to-date full-stack web application that helps make a seamless shift toward sustainable transportation by solving two crucial problems – the lack of charging infrastructure and range anxiety issues. As a result, ChargeSaathi provides a powerful solution for identifying charging stations, developing efficient routes, and cooperating with other drivers.

To achieve the goal, a map-based ecosystem is to be built in order to facilitate the location of EV charging stations and build up a social environment around them. In addition, ChargeSaathi uses the OpenStreetMap's Overpass API in order to collect open-source geographic data and allows filtering charging stations according to their connector types and charging speeds, as well as current availability.

From an architectural perspective, ChargeSaathi consists of a two-tier stack. First of all, the back-end uses the ASP.NET Core programming framework following MVC pattern and is connected to the NoSQL MongoDB database. The back-end makes intensive use of the .NET MemoryCache when requesting any external API. Meanwhile, the front-end is written using React 18 with Leaflet.

One of the most distinguishing features of ChargeSaathi is a special Social Sync module. Thanks to the Microsoft SignalR WebSockets library, it enables the two-way real-time communication where users can chat, share their map view ports, and report about any localized issues immediately. Moreover, the dual authentication flow involves JWT validation and Google OAuth.

Overall, ChargeSaathi connects infrastructure accessibility and driver awareness. Combining a geographical mapping system with real-time WebSocket communication and community-oriented reporting system, the project develops into a promising scalable technology

List of Figures

Fig 1.1 Gantt Chart 2
Fig 3.1 System Architecture 5
Fig 3.2.1 Physical ER Diagram 12
Fig 3.2.2 DFD Level 0 (Context Flow) 15
Fig 3.2.2.1 DFD Level 1 (Process Decomposition) 22
Fig 3.2.3.1 Use Case Diagram 25
Fig 3.2.3.2 Class Diagram 32
Fig 3.2.3.3 Sequence Diagram 35
Fig 3.2.3.4 Activity Diagram 42
Fig 3.3.1 State Transition Diagram 43
Fig 3.3.2.1 Login / Sign Up Screen 44
Fig 3.3.2.2 Home Screen 45
Fig 3.3.2.3 Chat Room Screen 46

List of Tables

Table 2.2 Comparison of Current Systems vs. Proposed System 6
Table 2.8 Hardware and Software Requirements 9

Abbreviations

API	Application Programming Interface
ASP	Active Server Pages (as used in ASP.NET)
DFD	Data Flow Diagram
ER	Entity-Relationship
EV	Electric Vehicle
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
BSON	Binary JSON
JSON	Java Script Object Notation
MVC	Model-View-Controller
NoSQL	Not Only SQL (Not in Relation, referring to the MongoDB database)
SQL	Structured Query Language
OAuth	Open Authorization (used for Google Sign-In)
OCPP	Open Charge Point Protocol
OCM	OpenChargeMap
OSM	OpenStreetMap
POI	Point of Interest
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
WS	WebSocket (referring to the SignalR communication protocol)
GPS	Global Positioning System
CORS	Cross-Origin Resource Sharing
CDN	Content Delivery Network
kW	Kilowatt
CCS	Combined Charging System

Table of Contents

Acknowledgement..... i

Abstract ii

List of Figures iii

List of Tables iv

List of Abbreviations v

Table of Contents vi

Chapter 1 Introduction to Project and Project Management..... 1

1.1 Project Summary 2

1.2 Purpose 3

1.3 Objective 4

1.4 Scope 5

1.5 Technology and Literature Review..... 6

Chapter 2 System Analysis..... 8

2.1 Study of Current System 9

2.2 Problem and Weaknesses of Current System 10

Chapter 3 System Design..... 11

3.1 System Design & Methodology 14

3.2 Database Design / Data Structure Design 16

3.3 Input / Output and Interface Design 17

Chapter 4 Conclusion and Future Enhancements..... 30

4.1 Overall Analysis of Project Viabilities 31

4.2 Problem Encountered and Possible Solutions 32

4.3 Summary of Project Work 34

4.4 Limitations 35

4.5 Future Enhancement 36

Appendix..... 40

References 42

CHAPTER 1

INTRODUCTION TO PROJECT AND PROJECT MANAGEMENT

1.1 PROJECT SUMMARY

ChargeSaathi is a modern, full-stack, state-of-the-art web application specifically tailored for Electric Vehicle (EV) drivers. The application provides an interactive, map-based interface that allows users to seamlessly locate EV charging stations globally. Utilizing open-source geolocation data alongside a robust real-time communication framework, ChargeSaathi transcends traditional static map applications by integrating community-driven capabilities. Key features include the extraction of map Points of Interest (POIs), "Social Sync" functionality for live driver-to-driver communication, dual-layer authentication, and user-submitted reporting systems.

1.2 PURPOSE

The transition from internal combustion engine vehicles to electric vehicles is accelerating, bringing forth a new challenge known as "range anxiety." EV drivers frequently struggle to find compatible, operational charging stations on long routes. The purpose of ChargeSaathi is to alleviate this anxiety by providing a highly specific, rapidly updating platform. It aims to foster a community of socially synced drivers who can find optimal charging locations while avoiding congested or broken charging stations.

1.3 OBJECTIVE

The primary objectives of the ChargeSaathi project are to:

Develop an intuitive, drag-responsive geographic map interface using React-Leaflet to display nearby EV stations based on user viewport bounds.
Implement an efficient backend caching mechanism using .NET MemoryCache to optimize traffic and prevent rate limits from external geographic APIs (Overpass API).
Establish a real-time, peer-to-peer WebSocket connection (Social Sync) using Microsoft SignalR to allow drivers to coordinate their routes via live chat and map viewport syncing.
Ensure robust data security through dual-factor authentication, utilizing both locally encrypted JWT tokens and seamless Google OAuth integration.

1.4 SCOPE

It is crucial to define the functional boundaries of the application to ensure the project remains focused and achievable.

What it can do:

Autonomously query and render EV station POIs from OpenStreetMap data upon map dragging or zooming.
Securely authenticate users using both local email/password flows and Google OAuth sign-in.
Allow users to persistently save specific charging stations to a personal "Favorites" list within a NoSQL database.
Enable real-time broadcasting of chat messages and map locations between active concurrent users on the platform.
Allow users to report broken or under-maintenance stations to a central database.
What it cannot do:

The system currently does not communicate directly with physical station hardware via OCPP (Open Charge Point Protocol), meaning it cannot show guaranteed real-time availability of individual plugs.
It does not feature a financial payment gateway or standard wallet for purchasing electricity at the station.
Hardware diagnostics of the electric vehicle itself are beyond the scope of this web application.

1.5 TECHNOLOGY AND LITERATURE REVIEW

Developing highly responsive map applications requires specialized tools that differ from standard web pages. Traditional tools like standard Google Maps API offer heavy monetization restrictions and less flexibility for custom clustering of niche data (like EV sockets). Consequently, this project leverages a high-performance, open-source stack.

1. Backend framework: ASP.NET Core 8.0 (C#) was selected for its mature handling of high-concurrency requests and native in-memory caching solutions.

2. Frontend framework: React 18, paired with react-leaflet, is used to maintain a dynamic Virtual DOM, critical for rendering hundreds of clustered map nodes without causing severe browser lag.
3. Data layer: MongoDB, a NoSQL data store, natively stores JSON documents, which is logically parallel to how map coordinate data (GeoJSON) is usually processed.

4. Real-time Protocol: Microsoft SignalR abstraction simplifies the raw WebSocket protocols, ensuring reliable connection drop-handling for mobile Web clients.

1.6 PROJECT PLANNING AND SCHEDULING

1.6.1 Project Development Approach and Justification

The project was developed utilizing the Agile Development Methodology. Rather than attempting to build the map, the backend, and the chat system all at once, the system was divided into bi-weekly sprints. This approach was justified because external integrations (like the Overpass API) are subject to strict downtime and rate limits; building iteratively allowed the team to safely test caching logic before building the React user interface on top of it.

1.6.2 Project Effort and Time, Cost Estimation

Effort and Time: The project required an estimated 14 weeks from initial concept to deployment. Major time allocations included 3 weeks for backend architecture, 4 weeks for the React Map UI, and 3 weeks resolving cache limits and Google Auth security flaws.

Cost Estimation: Development costs remained negligible natively. OpenStreetMap provides free raw data, the MongoDB Community Server is free to host locally, and development IDEs (VS Code / Visual Studio Community) are open-source. The primary theoretical cost involves remote cloud hosting for the IIS Server and the Node.js frontend runner.

1.6.3 Roles and Responsibilities

The project was executed by a two-person development team, allowing for a clear separation of concerns between the client-side interface and the server-side architecture.

Rudra Thakker: Frontend Developer (UI/UX & Client Logic)

- **Technologies:** React 18, React-Router, React-Leaflet, CSS3.
- **Responsibilities:**
    - Designed and implemented the core User Interface (UI), including the interactive map (Map.jsx), navigation sidebars (SearchFilter, RoutePanel), and the Social Sync chat window.
    - Configured the frontend integration of @react-oauth/google to handle secure authentication popups and token extraction.
    - Managed React State to dynamically render hundreds of clustered EV station markers without performance degradation on the client's browser.
    - Ensured the frontend correctly communicated with backend REST APIs and the SignalR WebSocket hub.

Devicharan Dasari: Backend Developer (API, Database & Real-Time Comms)

- **Technologies:** ASP.NET Core 8.0 (C#), MongoDB, Microsoft SignalR, .NET MemoryCache.
- **Responsibilities:**
  - Architected the Model-View-Controller (MVC) server-side application, developing secure RESTful endpoints for Authentication, Points of Interest (POI), and Interactions.
  - Designed and managed the NoSQL MongoDB schemas (Users, Favorites, ReportDetails).
  - Engineered the PoiController, implementing a custom HttpClient with strict 25-second timeouts and heavily optimized in-memory caching to fetch OpenStreetMap data without causing server thread hangs.
  - Built the SocialSyncHub using abstract WebSockets (SignalR) to broadcast real-time driver chat messages and map coordinates across the network.

1. Frontend / UI Developer: Responsible for setting up React, integrating the Leaflet map clusters, styling sidebar components, and configuring Google OAuth popups.

2. Backend Engineer / API Developer: Responsible for building the ASP.NET Core MVC APIs, writing the tight 25s timeout HttpClient logic for Overpass API, and configuring .NET MemoryCache.

3. Database & Integration Admin: Responsible for designing MongoDB schemas (User, Favorite, ReportDetail), securing JWT configurations, and coding the SignalR SocialSyncHub.

1.6.4 Group Dependencies

Successful module delivery was highly dependent on inter-component functionality. The React Map interface (Frontend) was blocked until the PoiController (Backend) successfully returned JSON coordinate arrays. Furthermore, the Social Sync chat interface fundamentally relied on the prior integration of the backend Auth Module to safely broadcast user usernames over Secure WebSockets.

1.7 PROJECT SCHEDULING

Gantt Chart:

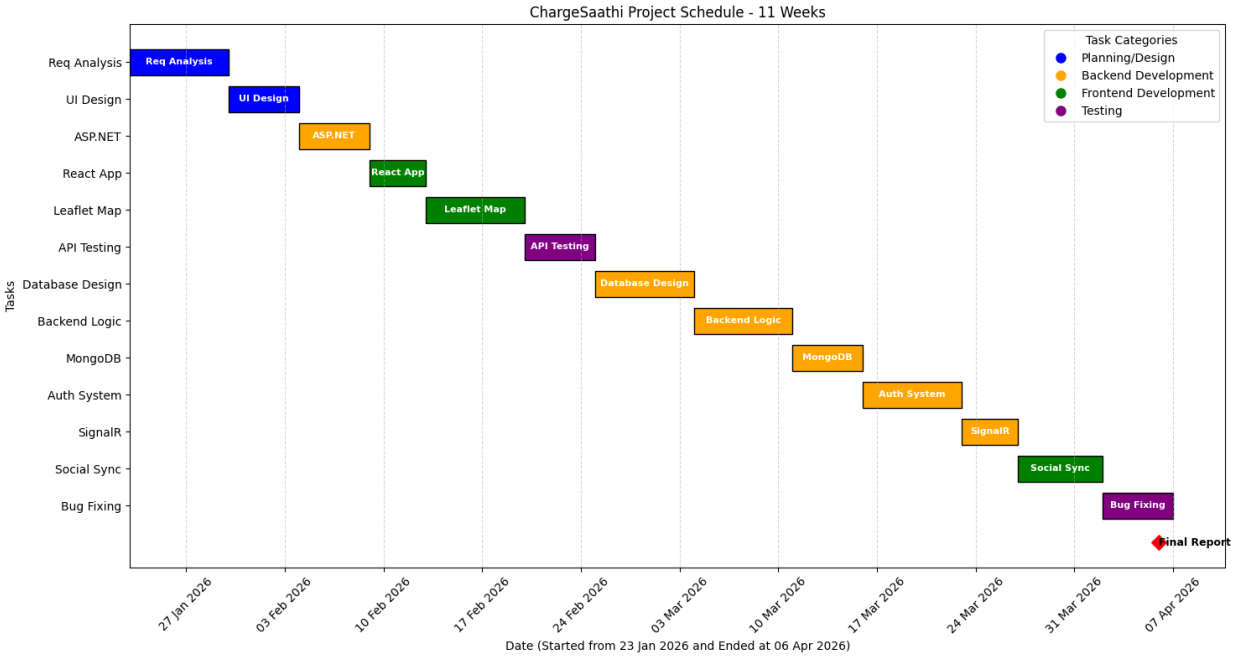


Figure 1.1 Gantt Chart

CHAPTER 2

INTRODUCTION TO PROJECT AND PROJECT MANAGEMENT

2.1 STUDY OF CURRENT SYSTEM

The current ecosystem for navigation relies heavily on generalized, proprietary mapping applications such as Google Maps, Apple Maps, or built-in vehicle navigation systems. In these current systems, Electric Vehicle charging stations are treated as standard points of interest (similar to gas stations or restaurants). Users typically input a destination, and the system attempts to overlay charging stations along the general route. Data updates regarding station operational status are managed centrally by the proprietary platform, often relying on algorithms that scrape web data or rely on delayed user reviews.

2.2 PROBLEM AND WEAKNESSES OF CURRENT SYSTEM

The reliance on generic navigation mapping creates a distinct set of problems specifically for the modern Electric Vehicle driver:

Lack of Specialization: Current systems often lump all "EV Chargers" together. They do not easily allow a user to aggressively filter the map based strictly on EV-specific hardware elements (e.g., distinguishing between a slow Level 2 AC charger and an ultra-fast DC CCS charger).
Static Interaction: Current maps do not offer live horizontal communication between drivers. If a station is out of order, or there is a massive line, a driver will not know until they physically arrive.
Proprietary Data Lag: Because data updates are handled centrally, broken stations can remain on the map for weeks.

Table 2.2 Comparison of Current Systems vs. Proposed System

Feature / Criteria	Standard Maps (e.g., Google Maps)	ChargeSaathi (Proposed System)
EV Station Filtering	Basic (Often lumps all chargers together)	Advanced (Filter by specific connector type and charging speed)
Real-Time Social Sync	None (Static map viewing)	Yes (Bidirectional chat and map-viewport syncing via SignalR)
Offline POI Caching	Proprietary algorithm	Implemented via .NET MemoryCache to prevent external API limits

User Reporting	Moderated, takes days to update	Instant peer-to-peer reporting for broken stations
Data Source	Closed-source proprietary	OpenChargeMap(OCM), OpenStreetMap (OSM) via Overpass API

2.3 REQUIREMENTS OF NEW SYSTEM

To solve these weaknesses, the new proposed system (ChargeSaathi) must meet the following strict requirements:

1. Dynamic Data Fetching: Must seamlessly pull geographical nodes from an open-source layer (OpenStreetMap) rather than a closed-source provider.

2. Specialized UI Filters: The interface must include sidebars to filter purely based on EV metrics (connector type, speed, network).

3. Live Extensibility: The system must feature a real-time, peer-to-peer WebSocket connection enabling drivers to chat, coordinate routing, and visually sync their map viewports with one another.

4. Aggressive Caching: To remain fast and avoid hitting third-party API rate limits, the system requires a highly intelligent local memory cache pipeline.

2.4 SYSTEM FEASIBILITY

A comprehensive feasibility study was conducted to ensure the project could be delivered efficiently.

2.4.1 Does the system contribute to the overall objectives of the organization?

Yes. The primary objective is to make sustainable EV commuting highly accessible and community-driven. By building a platform that pairs exact location logic with instant social updates, ChargeSaathi radically improves the confidence of EV drivers, directly contributing to the goal of eco-friendly technological adoption.

2.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints?

Yes. The project scope is highly feasible within the 11-week schedule by strict adherence to an Agile development approach. Fiscally, the system is exceptionally lean because it utilizes completely open-source tools: React 18, MongoDB Community Server, and the free-to-access Overpass API. Development tooling via ASP.NET Core and Visual Studio imposes zero licensing fees natively.

2.4.3 Can the system be integrated with other systems which are already in place?

Yes. The platform's backend operates purely via standardized RESTful JSON architectures and standard WebSockets. By relying on OAuth 2.0 (Google Authentication), it seamlessly integrates into user identities that individuals already utilize daily. Furthermore, its reliance on OpenStreetMap means it instantly integrates with the largest open geographic database in the world.

2.5 ACTIVITY / PROCESS IN NEW SYSTEM / PROPOSED SYSTEM

The functional process within ChargeSaathi operates via concurrent streams:

1. Map Initialization Activity: A driver accesses the web application and visually drags the map. The React frontend continually captures the geographic bounds (North, East, South, West cords).

2. POIs Resolution Activity: The frontend fires HTTP JSON requests containing bounds. The PoiController on the ASP.NET backend intercepts this. It checks its local .NET MemoryCache. If the bounds are cached, it returns them instantly. If missing, it dispatches an HTTP request to the external Overpass API, applying a strict 25s timeout to prevent thread blocks.

3. Community Hub Activity: A user clicks a broken station and clicks "Report." Almost simultaneously, they can open the "Social Sync" sidebar. They broadcast a message indicating the station is broken. The SocialSyncHub (SignalR) instantly pushes this message to the screens of all other concurrent drivers looking at that map region.

2.6 FEATURES OF NEW SYSTEM / PROPOSED SYSTEM

Dual-Layer Authentication Security: Allows users to log in traditionally via JWT tokens, or entirely bypass manual passwords using secure Google OAuth bindings.

"Social Sync" Real-Time Portals: Enables peer-to-peer driver chat and interactive map-focus features using Microsoft SignalR WebSockets.

Offline-First Memory Caching: Minimizes network latency and prevents Overpass API IP-bans by heavily utilizing local IIS/Kestrel Server memory.

React-Leaflet Clustered Interaction: Groups hundreds of raw geographic nodes into sleek, interactive numerical clusters that separate upon zoom.

2.7 LIST MAIN MODULES / COMPONENTS / PROCESSES / TECHNIQUES OF PROPOSED SYSTEM

The proposed architecture is categorized into the following distinct development modules:

Authentication & Identity Module (AuthController.cs, React Auth Logic)
Station Locator & Data Module (PoiController.cs, Overpass API extraction)
Real-Time Social Sync Module (SocialSyncHub.cs, SignalR WebSocket pipeline)
Interaction Module (InteractionController.cs, handling Favorites and Reports logic against MongoDB)
Interactive Map Interface UI Component (React Virtual DOM, Leaflet maps)

2.8 SELECTION OF HARDWARE / SOFTWARE / ALGORITHMS / METHODOLOGY / TECHNIQUES

ChargeSaathi selects the MERN/.NET Stack hybrid methodology. Rather than processing Node.js backend logic, ASP.NET Core was selectively chosen because its deeply optimized multithreading and garbage collection techniques handle HTTP routing timeouts and In-Memory caching substantially better than standard Express.js setups. MongoDB was heavily favored over standard relational SQL because geographic map nodes natively return JSON coordinate variants, aligning perfectly with a NoSQL document paradigm.

Table 2.8 Hardware and Software Requirements

Category	**Requirement Specification**
Hardware (Server)	Dual-Core CPU, 4GB RAM, 20GB Storage, Active Internet Connection
Hardware (Client)	Any standard Smartphone or PC with 2GB+ RAM
OS Environment	Windows 10/11, macOS, or Linux
Backend Framework	ASP.NET Core 8.0 (C#)
Frontend Framework	React 18 (Node.js Environment)
Database	MongoDB Server (Atlas Cloud)
Development Tools	Visual Studio 2022 / VS Code, Postman (API Testing)

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM DESIGN & METHODOLOGY

The system design of the ChargeSaathi application is structured around a distributed Client-Server architecture heavily utilizing the MVC (Model-View-Controller) pattern on the backend and a Component-Based Architecture on the frontend.

The methodology adheres to a micro-services-inspired separation of concerns:

1. Frontend (View): The React layer strictly manages the UI and the Leaflet Virtual-DOM. It handles no heavy computation, only state management and rendering via React Hooks.

2. Backend (Controller/Services): ASP.NET Core manages the heavy lifting. All external API routing (Overpass API) and WebSockets logic (SignalR) are encapsulated in distinct C# Services.

3. Data (Model): Business logic entities (User, Favorite, Report) are strongly typed via C# Models and serialized gracefully into BSON for the MongoDB driver.

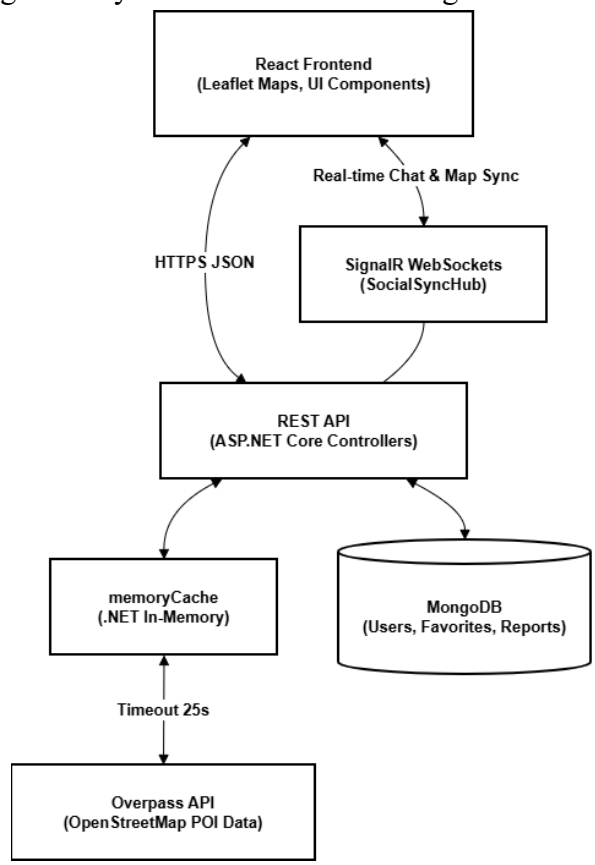


Fig 3.1 System Architecture

3.2 DATABASE DESIGN / DATA STRUCTURE DESIGN / PROCESS DESIGN

Because geographic coordinate extraction natively results in nested JSON nodes, traditional relational SQL databases are inefficient for this task. Therefore, the system incorporates MongoDB, utilizing a schema-less NoSQL data structure.

Database Modality
The primary collections within the database function autonomously but are linked logically via the UserId string.

1. Users Collection: Stores Google OAuth IDs, securely salted password hashes, emails, and usernames.
2. Favorites Collection: Links a specific map coordinate to a user's ID.
3. Reports Collection: Maintains records of user-flagged broken stations.

3.2.1 Entity-Relationship (ER) Design

The physical mapping of the NoSQL structure mimics a one-to-many relationship where one User Node can logically expand to multiple Favorites and multiple Reports.

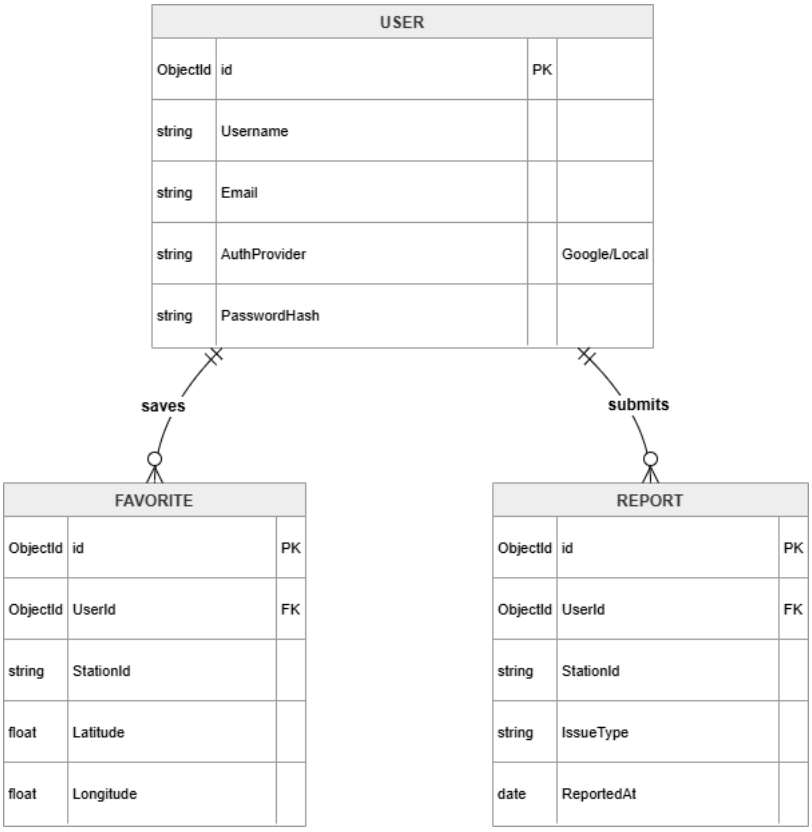


Fig 3.2.1 Physical ER Diagram

3.2.2 Process Design & Data Flow (DFD)

The procedural flow of the application ensures that raw client requests never directly hit the OpenStreetMap network, acting as an essential proxy. The Level 0 DFD illustrates the macro relationship between the Driver, the ChargeSaathi System, and the external data repositories.

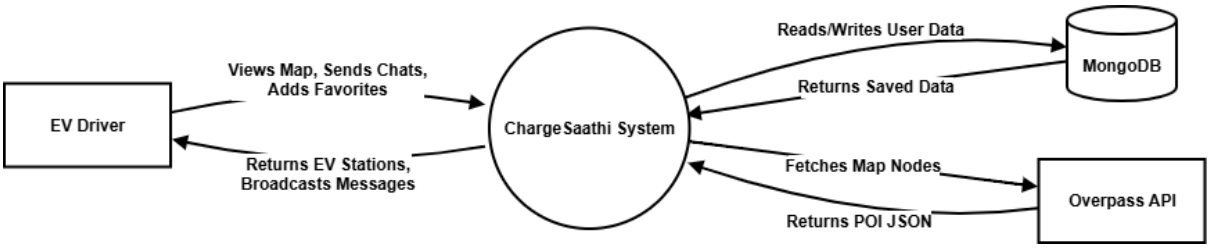


Fig 3.2.2 DFD Level 0 (Context Flow)

The Level 1 DFD decomposes this into the granular sub-processes: Authentication validation, POI bounding mechanisms, SignalR chat syncing, and report handling.

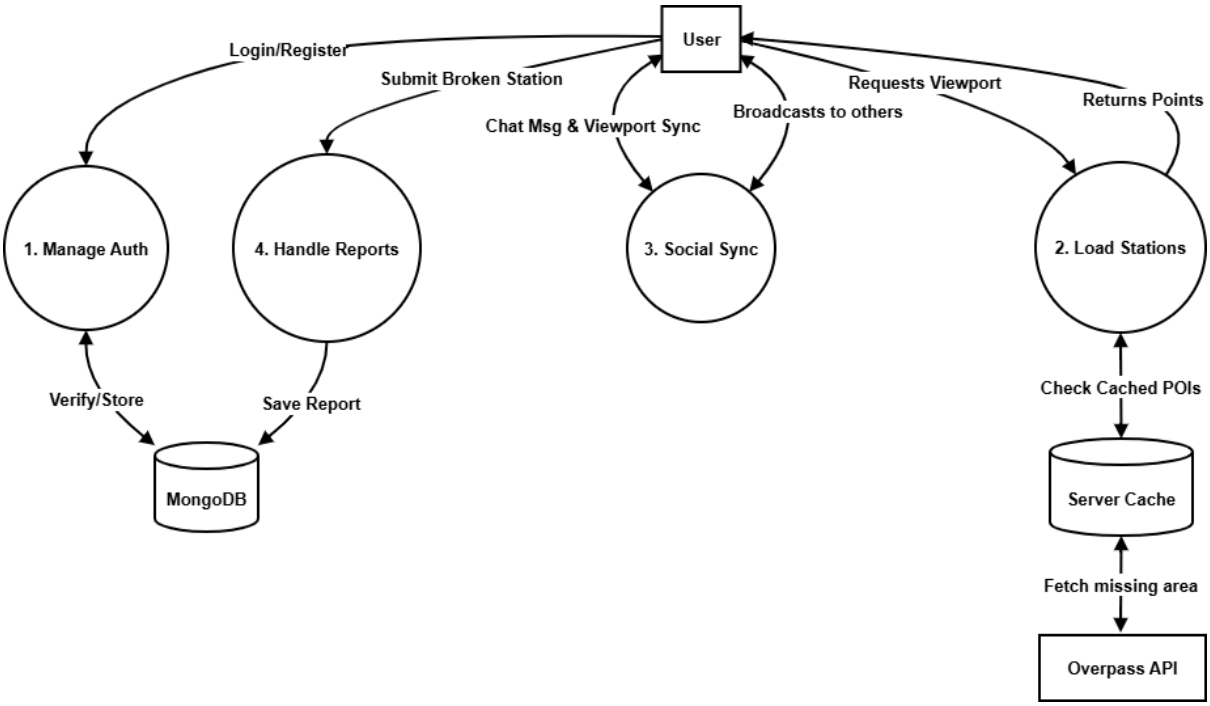


Fig 3.2.2.1 DFD Level 1 (Process Decomposition)

3.2.3 Behavioral Modeling (UML Diagrams)

To ensure the system logic handles asynchronous user actions cohesively, several Unified Modeling Language diagrams map behavioral expectations.

Use Case Modeling

Maps the internal scopes of the driver interacting with secondary API actors.

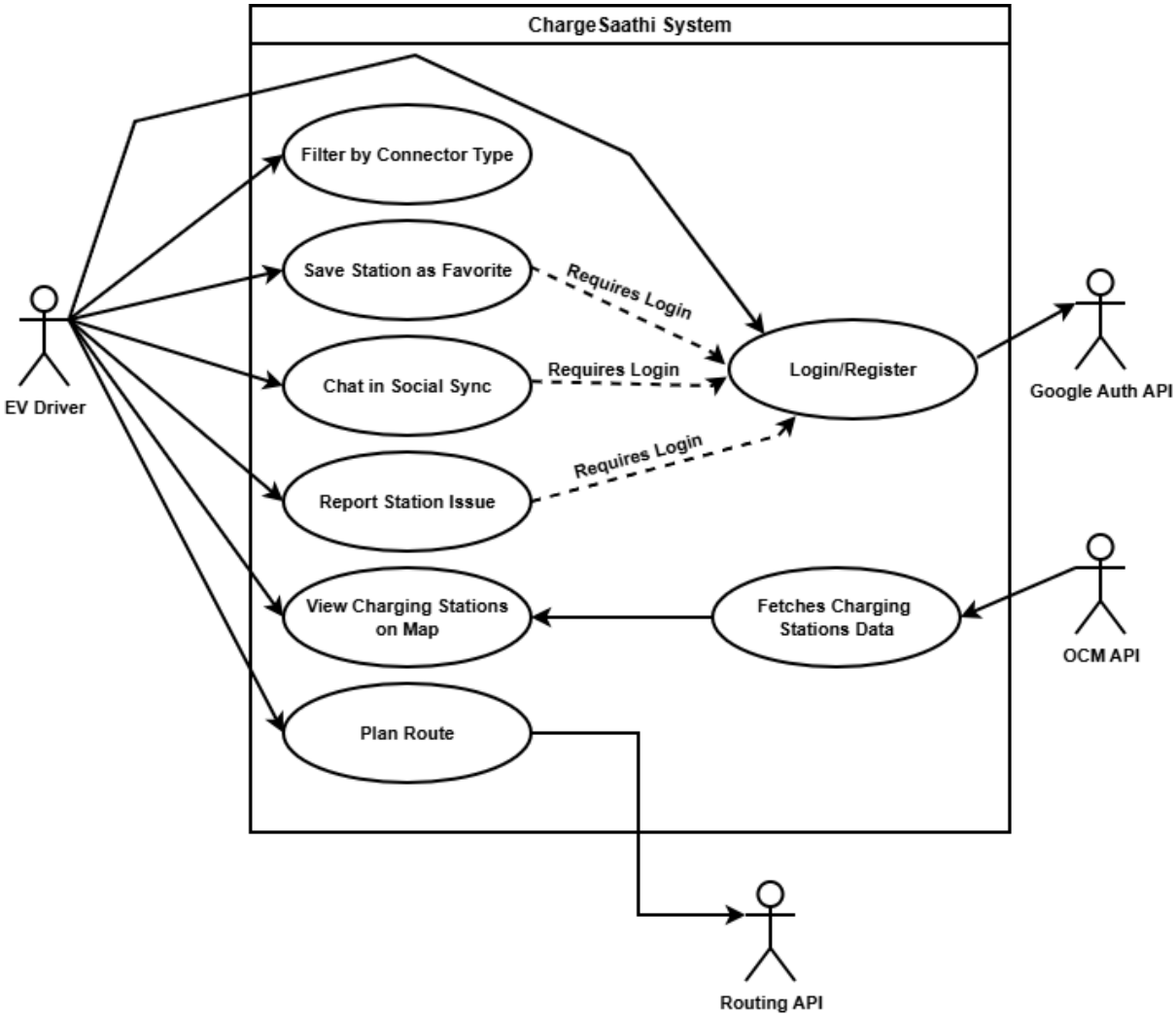


Fig 3.2.3.1 Use Case Diagram

Class Structuring

Details the C# object-oriented paradigms.

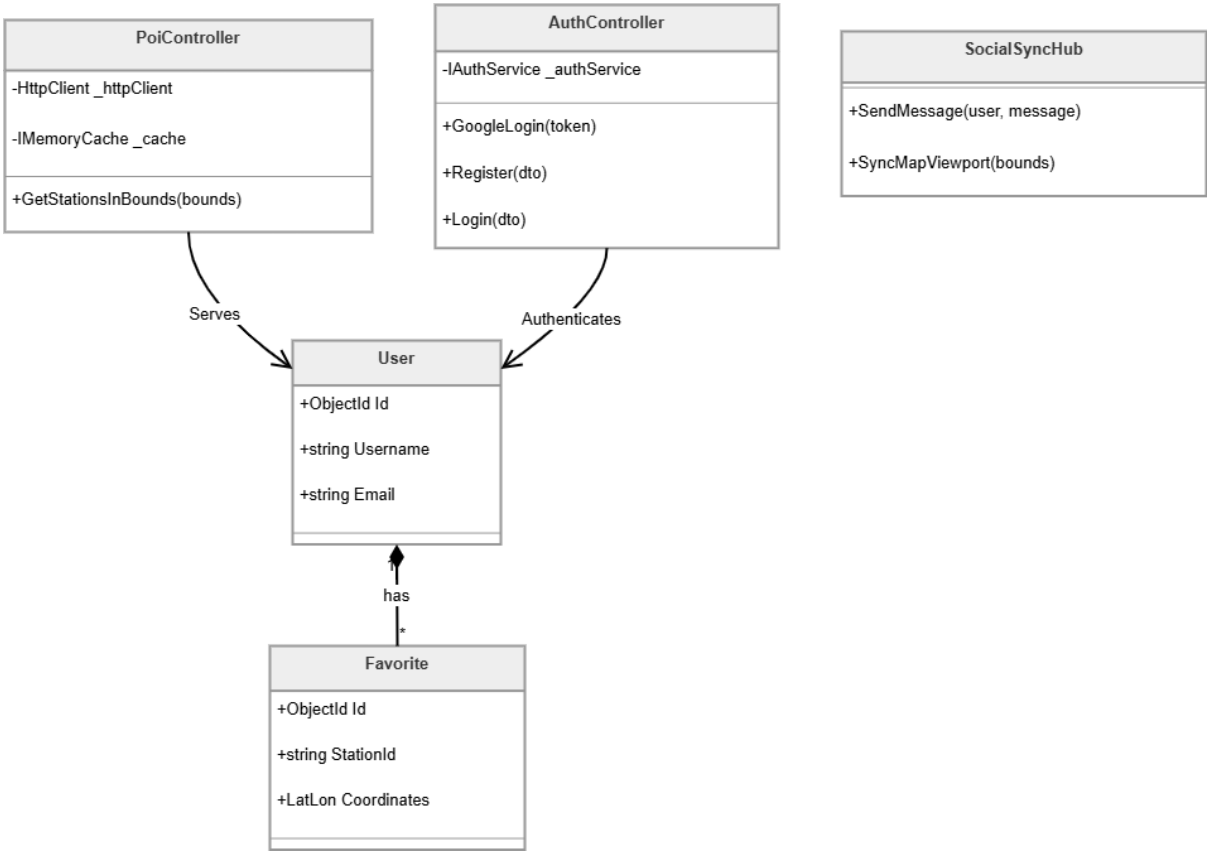


Fig 3.2.3.2 Class Diagram

Sequence Operations

Follows the exact chronological HTTP steps during an Overpass API data fetch.

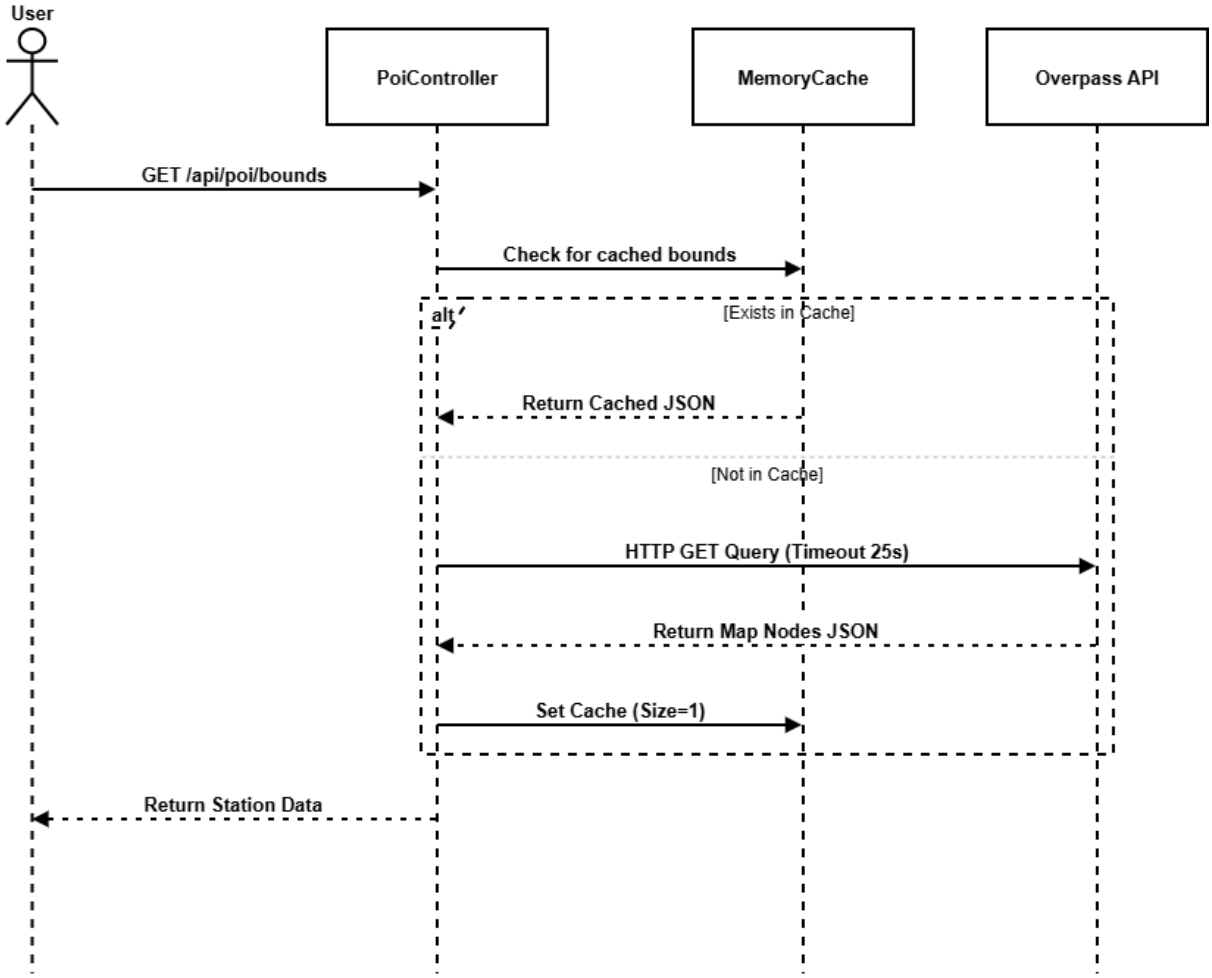


Fig 3.2.3.3 Sequence Diagram

Activity Flow

Demonstrates the end-to-end user journey from opening the application to plotting a route.

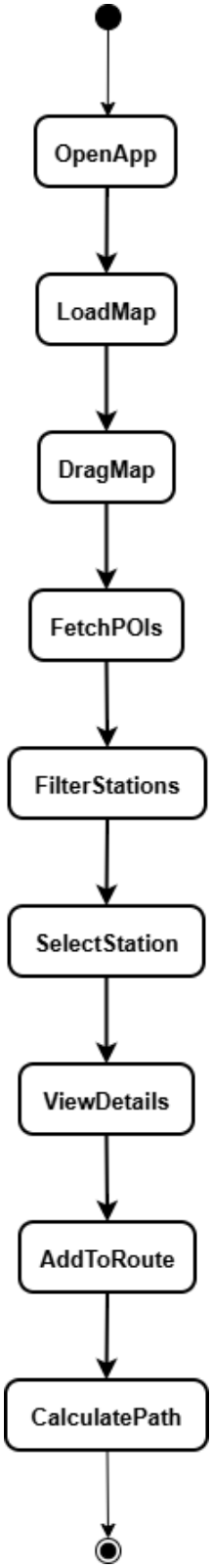


Fig 3.2.3.4 Activity Diagram

3.3 INPUT / OUTPUT AND INTERFACE DESIGN

3.3.1 State Transition Diagram (SignalR Social Sync)

The input/output behavior of the real-time chat interface depends entirely on connection states. The system dynamically transitions the interface based on WebSocket handshake validations (e.g., transitioning from 'Connecting' to 'Syncing Viewport' or 'Disconnected').

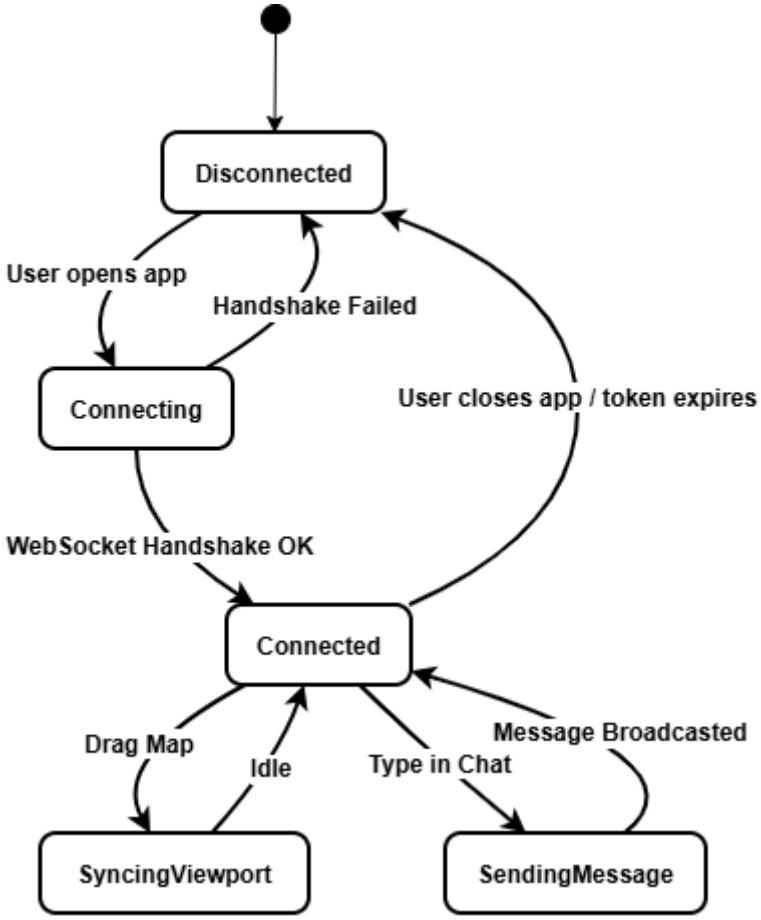


Fig 3.3.1 State Transition Diagram

3.3.2 Samples of Forms, Reports, and Interface

The primary User Interface inputs revolve around map interactivity rather than traditional text forms.

Map Interface Engine (Map.jsx): Input is drawn from the user's cursor or touch metrics dragging the map. Output is a visual cluster of EV station nodes dynamically rendering in real-time.

Dual Login Interface (Auth.jsx): Allows input form fields manually or an invisible standard output token-exchange via a Google Sign-In Popup widget.

Sidebars (RoutePanel / SearchFilter): CSS-styled HTML inputs outputting specific query parameters to the backend to selectively display UI elements.

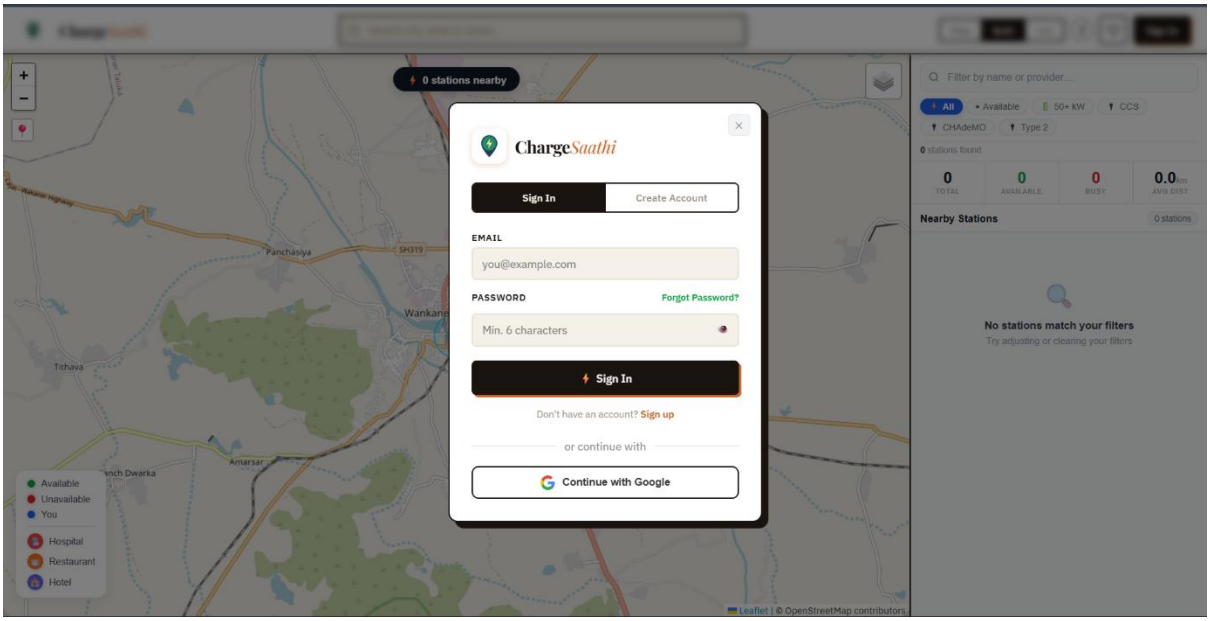


Fig 3.3.2.1 Login / Sign Up Screen

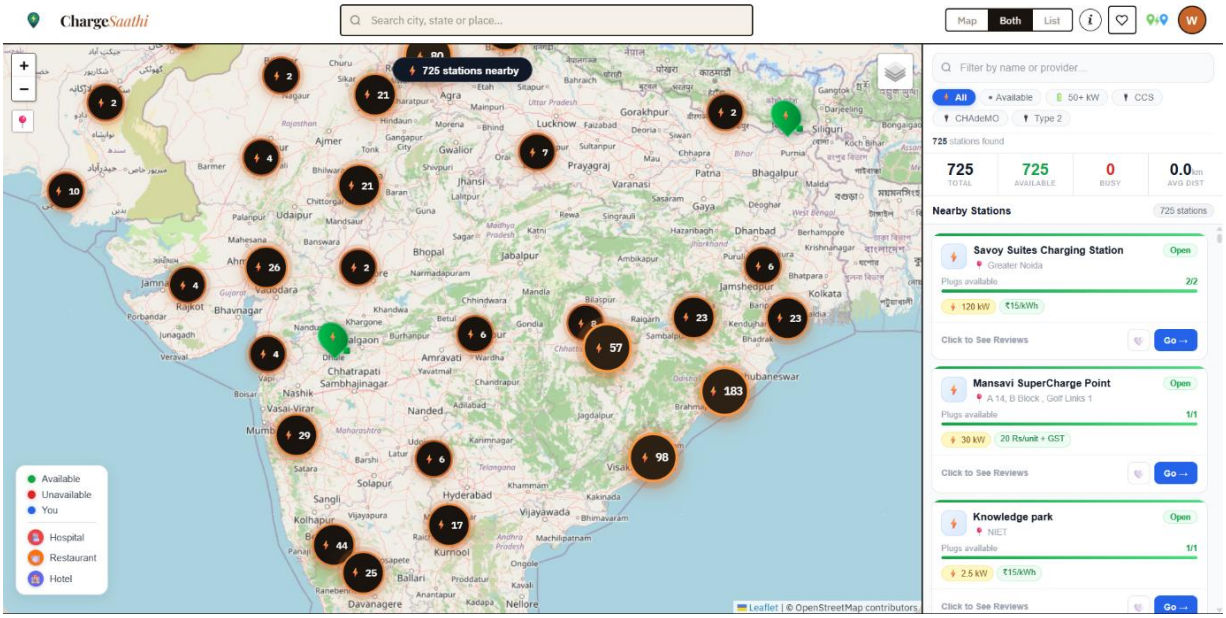


Fig 3.3.2.2 Home Screen

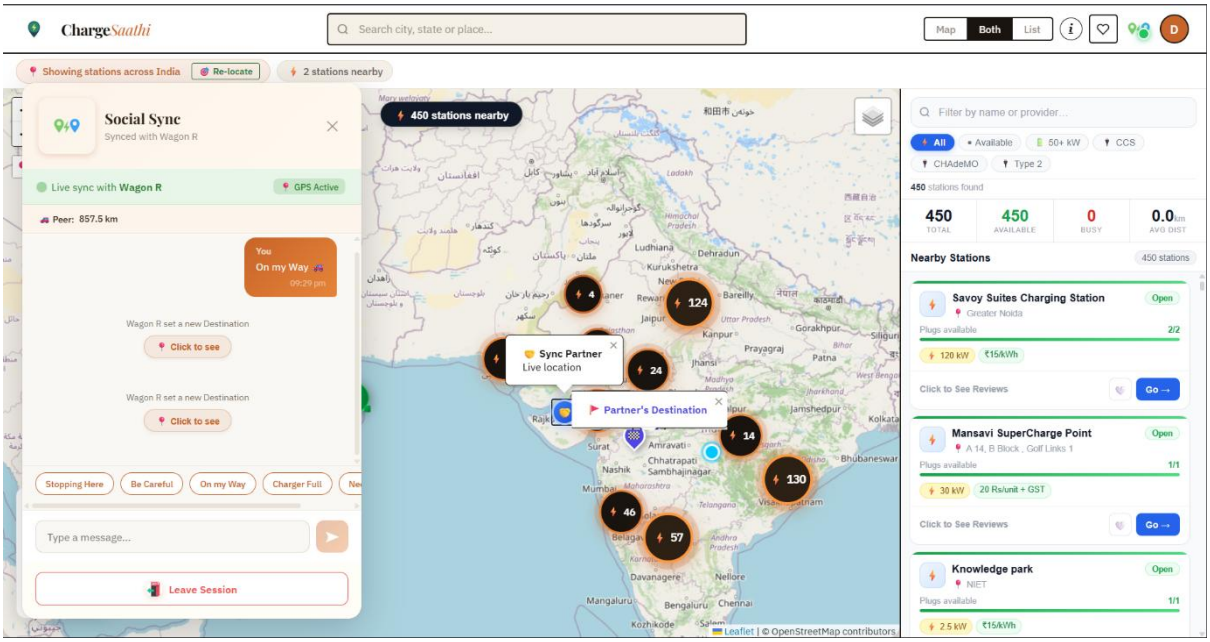


Fig 3.3.2.3 Chat Room Screen

3.3.3 Access Control / Mechanism / Security

Access control mechanisms form the backbone of the application's stability.

JSON Web Token (JWT) Security: Once authenticated, users are issued an encrypted JWT containing specific payload claims (Expiration limits, UserId). The frontend must send this token in the Authorization: Bearer header. Any missing or modified tokens result in an immediate 401 Unauthorized.

OAuth Boundaries: Users authenticating via Google OAuth have their emails appended internally to the MongoDB. A strict mechanism prevents users with Google-linked accounts from utilizing manual password reset flows.

Cross-Origin Resource Sharing (CORS): The ASP.NET server implements a strict middleware CORS policy. It forcibly rejects HTTP and WebSocket requests not exactly matching the AllowReactApp origin parameter (e.g., generic local hosts).

CHAPTER 4

CONCLUSION AND FUTURE ENHANCEMENTS

4.1 OVERALL ANALYSIS OF PROJECT VIABILITIES

The overall viability of the ChargeSaathi EV Application is remarkably high. As global automotive markets pivot intensely toward electric vehicles, the demand for specialized, dedicated infrastructure tracking solutions outpaces current generalized map systems. By leveraging a highly scalable technology stack—specifically MongoDB for rapid, schema-less geospatial JSON storage, and ASP.NET Core for asynchronous routing—the application is fundamentally built for high-concurrency production usage. The integration of community-driven features like Social Sync elevates the platform from a simple utility to an active peer-to-peer ecosystem, securing its viability in an expanding modern market.

4.2 PROBLEM ENCOUNTERED AND POSSIBLE SOLUTIONS

Throughout the rigorous 14-week development cycle, several significant technical challenges were encountered and subsequently resolved:

Overpass API Thread Hanging:
Problem: The external geographical provider (OpenStreetMap via Overpass API) occasionally suffered from latency. The ASP.NET server would hold threads open waiting for a response, ultimately causing server-wide hangs.
Solution: Implemented a strict, custom 25-second HTTP timeout boundary and deployed .NET MemoryCache to cache localized bounds, drastically reducing outward network calls.
.NET MemoryCache Size Limit Crashes:
Problem: An arbitrary SizeLimit was initially applied to the memory cache without defining sizing metrics per entry, resulting in runtime InvalidOperationException errors and HTTP 500 crashes during map loading.
Solution: Recalibrated the caching logic to assign Size = 1 integers to each geographic query object, allowing the server to correctly parse and eject cache memory safely.
Authentication Routing Flaws:
Problem: Users authenticated exclusively via Google OAuth were theoretically able to access standard email/password reset routes, creating a logic flaw.
Solution: Implemented strict backend routing logic that identifies and blocks AuthProvider metadata to restrict Google-linked accounts from manual system resets.

4.3 SUMMARY OF PROJECT WORK

The completion of this mini-project marks the successful deployment of a full-stack, distributed web architecture. The backend servers were engineered in C# (.NET 8) to securely handle identity validation, WebSocket broadcasting via Microsoft SignalR, and geographic API proxying. Simultaneously, the frontend was constructed using React 18 and React-Leaflet to dynamically render clustered EV stations onto a responsive UI. The project succeeded in tying these functional modules together, ultimately creating an elegant mapping solution enhanced by real-time driver chat and instant coordinate serialization to a NoSQL database (MongoDB).

4.4 LIMITATIONS

While technically sound, the current iteration of the system faces several inherent limitations:

1. Data Source Dependency: The platform relies entirely on the open-source OpenStreetMap (Overpass API) architecture. The accuracy of EV station locations is dependent on localized community mapping; if OSM data is outdated in a specific rural region, the application cannot inherently know.

2. Lack of Hardware Protocol Integration: The system plots the geographic location of charging infrastructure but does not communicate physically with the charging hardware. Therefore, it cannot guarantee if an individual plug at a station is currently occupied or unplugged.

3. Social Sync Capacity Limit: Currently, the real-time "Social Sync" module is architected to handle peer-to-peer communication limited to only 2 persons at a time (a strict 1-on-1 connection).

4. Browser Limitation: While highly responsive, it is currently a web-based application requiring a browser, rather than a natively compiled iOS/Android mobile architecture.

4.5 FUTURE ENHANCEMENT

To evolve the system from a localized academic project into an enterprise-ready product, several future enhancements are planned:

1. Open Charge Point Protocol (OCPP) Integration: The most vital future step is connecting the backend directly to standard OCPP networks. This would allow the map to display "Live Availability Status" glowing green or red depending on if a vehicle is actively plugged into the hardware.

2. Native Mobile Application Expansion: Migrating the React 18 browser codebase to a React Native or Flutter framework to deploy the application directly to the Google Play Store and Apple App Store, allowing for native push notifications and background GPS logic.

3. Social Sync Group Expansion: Upgrading the Microsoft SignalR Hub logic to expand the Social Sync chat limitation. Instead of a 2-person peer-to-peer limit, the future system aims to support larger driver lobbies or group map-syncing (e.g., 10+ drivers coordinating a group road trip in real-time).

4.6 PROJECT OUTCOMES AND DISCUSSION

The ultimate outcome of the ChargeSaathi project proves that real-time, peer-to-peer WebSockets can interact seamlessly alongside computationally heavy map-rendering operations without overloading client browsers. The project achieved its core objective: mitigating range anxiety for Electric Vehicle drivers. By fostering a digital environment where drivers can visually locate stations, permanently favorite their regular routes, and engage in live communication with peers simultaneously navigating the same map view, the project successfully bridges the gap between static geographic tracking and a specialized driver social network.

References

[1] Open Charge Alliance, "Open Charge Point Protocol (OCPP) Version 2.0.1," *Open Charge Alliance*, [Online]. Available: https://www.openchargealliance.org/.

[2] Open Charge Map, "The global public registry of electric vehicle charging locations," *Open Charge Map Foundation*, [Online]. Available: https://openchargemap.org/site/develop/api.

[3]PlugShare, "PlugShare is a free EV driver's app for iOS, Android, and web, allowing users to find charging stations, leave reviews, and connect with other plug-in vehicle owners.," *Plug Share* [Online]. Available: https://www.plugshare.com/

Regular Report Diary

Invention Disclosure Form

1. **What is the title of the invention?**

Mr./ Ms/Dr.	Name (Full)	Department	Designation	Mobile No.	Email	Postal Address
Mr	Devicharan Dasari	CE – B Tech	Student	9586629899	devicharandasari019@gmail.com	Bhatiya Society, Near Jalaram Mandir, Wankaner , Gujarat 636321
Mr	Rudra Thakker	CE – B Tech	Student	7802049505	rudradilipthacker@gmail.com	17 VADI, ADIPUR, Gandhidham, PO:Adipur, DIST: Kachchh, Gujarat, 370205
Mr	Viraj Daxini	CE – B Tech	Assistant Professor	9427270518	viraj.daxini@marwadieducation.edu.in	Marwadi University, Rajkot Morbi Highway RAJKOT, GUJARAT 360003

2. **Provide an abstract or summary of the invention (in 100 words or less).**

ChargeSaathi is a web platform that simplifies the electric vehicle (EV) charging experience. It helps users find and navigate to nearby charging stations through easy-to-use map and list views. Key features include real-time availability of stations, detailed information about each station (such as power output, connector types, and pricing), and listings of local amenities. Users can keep a personalized list of their favorite stations, submit reviews and ratings, and report any operational issues. By gathering updates from users and centralizing important charging information, the platform makes EV route planning easier and improves access to infrastructure for drivers.

3. **What problem does the invention solve?**
(Explain the existing problem(s) and limitations in detail.)

ChargeSaathi is a solution that addresses the problem of "Range Anxiety" faced by Indian electric vehicle owners. It is estimated that the Indian market is expected to reach over 5.5 million electric vehicles by the year 2025; however, the lack of an accurate real-time electric charging station locator tool is present in Indian Tier-2 cities such as Rajkot, Gujarat. The current tools have significant drawbacks: "Open Charge Map" shows outdated information with no routing or point-of-interest support; "PlugShare" is limited in Indian cities and relies on the Google Places API as the source of location information, which in turn redirects users to Google Maps for routing, thus breaking the app experience; "Tata EZ Charge" is restricted to Tata network stations only, thus showing vendor lock-in. None of the tools show nearby point-of-interest information (such as restaurants or hotels), which helps in understanding the waiting time at the charging point; they cannot be used to filter charging points by connector type or power type; nor can they be used to show community-sourced availability with user accountability.

ChargeSaathi helps to mitigate these concerns with its inclusion of a self-contained map, created with Leaflet.js, which provides access to four tile layer options, real-time electric vehicle station information from OpenChargeMap, nearby points of interest with Overpass/OSM, and turn-by-turn directions from GraphHopper, all without relying on Google Maps. Additionally, **ChargeSaathi** provides its users with Social Sync, which allows for sharing of real-time locations, route progress, point of interest stop information, and quick messaging between two traveling electric vehicles. The application also provides its users with a guest-first access approach, whereby users can access key features without needing to sign up, but community-focused features require users to sign up. All of these features make **ChargeSaathi** an all-inclusive, specially designed electric vehicle travel companion, specifically for users in India.

4. **What is the general utility or application of the invention?**
(Domains, use cases, and beneficiaries.)

ChargeSaathi is a complete full-stack web application for discovering Electric Vehicle charging stations. It covers different domains such as Electric Vehicle Infrastructure, Smart City Navigation, Location-Based Services, and Community-Driven Data Platforms. Salient use cases involve the discovery of Electric Vehicle charging stations in real time, provision of Points of Interest (POI) overlay during charging station wait times, routing, community-based Electric Vehicle charging station availability, and Social Sync for collaborative Electric Vehicle travel. The target audience for the application consists of daily Electric Vehicle commuters, long-haul Electric Vehicle travelers, Electric Vehicle fleet operators, and Electric Vehicle infrastructure planners, with a particular focus on Tier-2 Indian cities.

5. **What advantages does the invention provide in terms of increased efficiency or efficacy compared to existing solutions?**

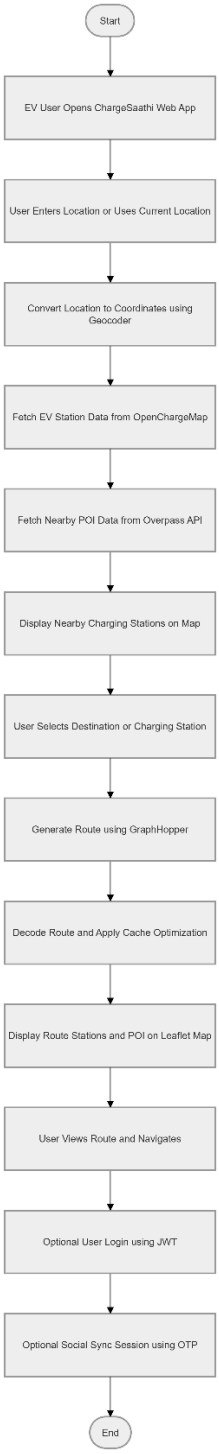
- Self-Contained Map : This feature allows for a completely integrated Leaflet.js-based map containing four tile layers. This eliminates Google Maps redirect issues, unlike PlugShare, whose location and navigation services are based on Google Places API.
- Multi-Network Support : This feature allows for compatibility with all electric vehicle charging networks via OpenChargeMap. This is unlike Tata EZ Charge, whose services are limited to Tata stations only.

- **Social Sync : This is a new concept for a real-time collaborative electric vehicle travel session based on OTP technology. This allows two users to share live location, route progress, point-of-interaction halt information, and quick messaging. This feature is not available on other electric vehicle platforms.**

6. **What is the best way of using the invention, and what are the possible variants?** (Medical, industrial, military, child/elder, adventure, etc.)

The best usage scenario would involve the operator of an Electric Vehicle (EV) service using ChargeSaathi before or in the course of the journey to locate the nearest compatible charging station, plan the route to the station, determine the nearby points of interest based on the expected time to be spent, and share the planned route using Social Sync with a companion. Variations to the application would involve a fleet management version for EV operators running a fleet of taxis or logistics vehicles, and a version for highway travel with charging stops.

7. **Explain the working of the invention with drawings, schematics, or flow diagrams, including complete explanations.**



ChargeSaathi provides the capability for an EV user to open the ChargeSaathi web application to begin charging their electric vehicle. The user can either enter a specific destination or use their current location; in either case, ChargeSaathi will leverage a geocoding service to translate the entered location into geographic coordinates that will allow for data retrieval related to available EV charging stations through OpenChargeMap (as well as leveraging Overpass API in order to retrieve available Points-of-Interest in relation to the charging stations selected).

Once available charging stations have been displayed on a map, the user may choose to select a destination or an EV charging station. In so doing, a routing engine will calculate the optimal route from the current position to the selected destination EV charging station and cache it (in order to improve routing performance). The map display incorporates the route displayed on the map; the EV charging stations located along the route; and all of the Points-of-Interest near all of the EV charging stations along the route (included with the Points-of-Interest). Optionally, the user may choose to enable additional capabilities such as favorite stations and/or Social Sync before proceeding with the navigation process.

8. **Have you conducted a primary patent search? If yes, provide details of the relevant prior patents.**

A primary patent search was performed at the IP India Public Search Portal (https:/iprsearch.ipindia.gov.in/PublicSearch) to find current patents that exist for EV charging systems. The search returned one relevant patent titled "Web-Based Smart EV Charging System with Slot Pre-Booking, OTP Authentication, Location Tracking and Automated Billing". This previous patent is primarily designed for charging slot reservation, user authentication and billing automation. The current invention (i.e. ChargeSaathi) differs from this previous patent, as ChargeSaathi emphasises integrated EV charging station discovery, routing, POIs (Point Of Interest) visualisation, and self-contained navigation, whereas the previous patent relates to only slot reservation and billing systems for EV chargers. Thus, while the two patents relate to EV chargers, the previous patent does not disclose the integrated navigation and mapping architecture specifically used in the current invention.

9. **What is the existing state-of-the-art and prior art related to this invention?** (Brief background of current products/processes in the market.)

The current electric vehicle (EV) charging station locator tools include PlugShare, Tata EZ Charge, and OpenChargeMap. These tools provide information about EV charging stations; however, these tools rely heavily on external routing services such as Google Maps. The tools provided by individual vendors allow users to access specific charging stations, while the tools provided by individual vendors lack routing capabilities along with points of interest. The majority of the tools lack the feature of providing users with a self-sustaining system. The proposed invention, "**ChargeSaathi**," will overcome the shortcomings of the current tools by providing users with the facility of discovering EV charging stations along with routing capabilities.

10. **What are the known ways others have tried to solve similar problems, and what are the drawbacks of those approaches?**
(Include prior art documentation and how your invention overcomes these drawbacks.)

The existing solutions, such as Google Maps, PlugShare, and Tata EZ Charge, provide access to electric vehicle charging stations via third-party navigation tools. These applications rely to a great extent on proprietary APIs and often require the user to access alternative applications for routing. These applications, based on proprietary infrastructures, provide access to a limited subset of charging stations. However, open data sources do not provide routing or point-of-interest information. Hence, the proposed invention will provide a complete solution to overcome the limitations.

11. **List the technical features and elements of the invention with a complete start-to-end description.**

A web-based Electric Vehicle Charging Station Locator incorporates modular systems that include a React-based front-end implementation, and a back-end consisting of OpenChargeMap EV Station data, and Overpass API data for nearby points of interest. The front-end uses JavaScript functionality to provide map layers (via Leaflet.js), and location markers (representing charging stations) as well as some limited route calculations (via GraphHopper services) based on a user-selected starting point and destination. SemaphoreSlim is used in the back-end to govern the number of API calls to the two APIs as well as to cache API responses (via IMemoryCache with time-out values) to achieve maximum efficiency for the overall system. User input for locations are processed by the back-end, which generates the necessary routing information, and presents the results on the map interface in real-time (dynamically).

12. **Which features of your invention are novel and how do they distinguish it from the closest existing technologies?**

Disclosed is a self-contained, unified EV navigation system that comprises charging station discovery, routing and point-of-interest visualization into one single interface. The existing EV Locating Systems have traditionally relied on external navigational systems to provide redirections. This EV Navigation System utilizes the internal processes to perform the routing and visualisation of map information.

The combination of these features results in an overall enhancement of performance, reliability and usability over conventional EV locating platforms.

The newly proposed Social Sync will allow for establishing temporary collaborative sessions using OTPs (One-Time Passwords) so that multiple electric vehicle users can share their routes in real-time; update their locations; and receive notifications about POIs (Points of Interest) along the route. Such functions are not available in traditional systems used to navigate electric vehicles.

13. **Has the invention been built, tested, or implemented?**
    If yes, provide efficiency/efficacy details.

    The invention has been developed as a functioning prototype and is at present in an ongoing test phase. Core capabilities such as finding electric vehicle charging stations, generating routes, and visualizing nearby points of interest have all been successfully built and tested internally. Furthermore, preliminary testing showed that the individual modules of the system have been operating reliably, have integrated correctly with each of the external APIs, and that they are functioning successfully when they are connected to each other. Although the system has not been released to the general public, we are continuing to validate its functionality and to refine its design.

14. **When and how was the invention first conceived?**
    (Origin of idea, inspiration, and evolution.)

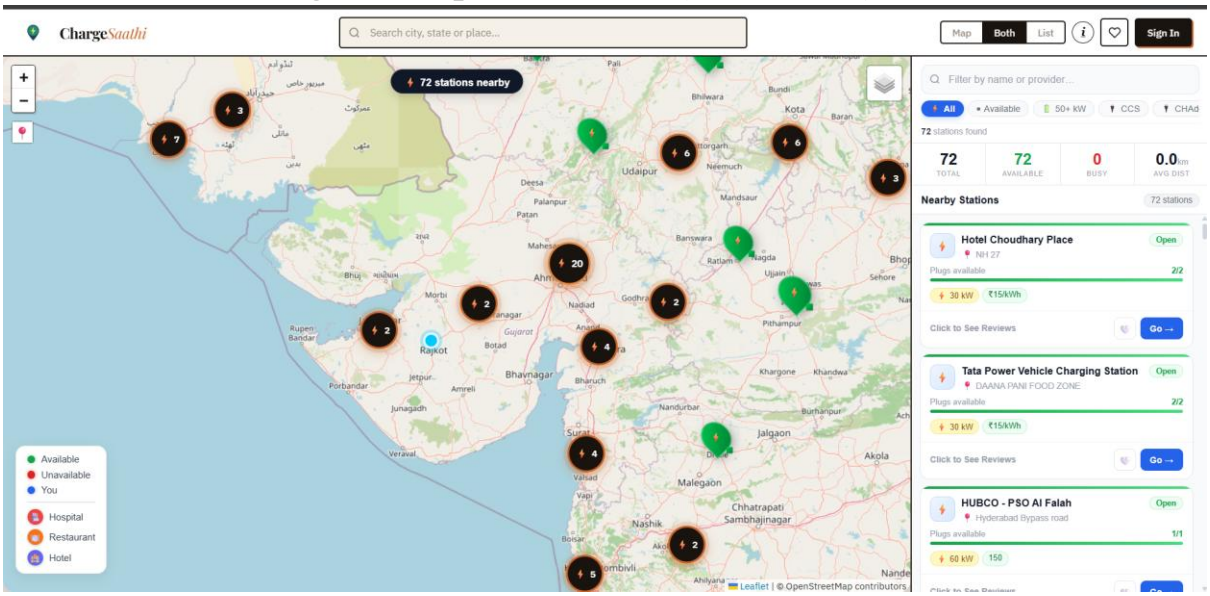
    The idea to invent such a device began in January 2026 when we were drafting out a small project for school; I first thought about the lack of assistance for EV (electric vehicle) users when trying to find available charging stations within tier two cities. There were numerous workshops where everyone discussed how to eliminate our reliance on third party navigation systems and make accessible. The device was built using an iterative cycle of design and testing over many (7) development sprints from January to April 2026, working with four separate groups.

15. **Has the invention been sold, publicly used, disclosed, or published earlier?**
    If yes, provide dates, disclosure details, NDAs, and filing deadlines.

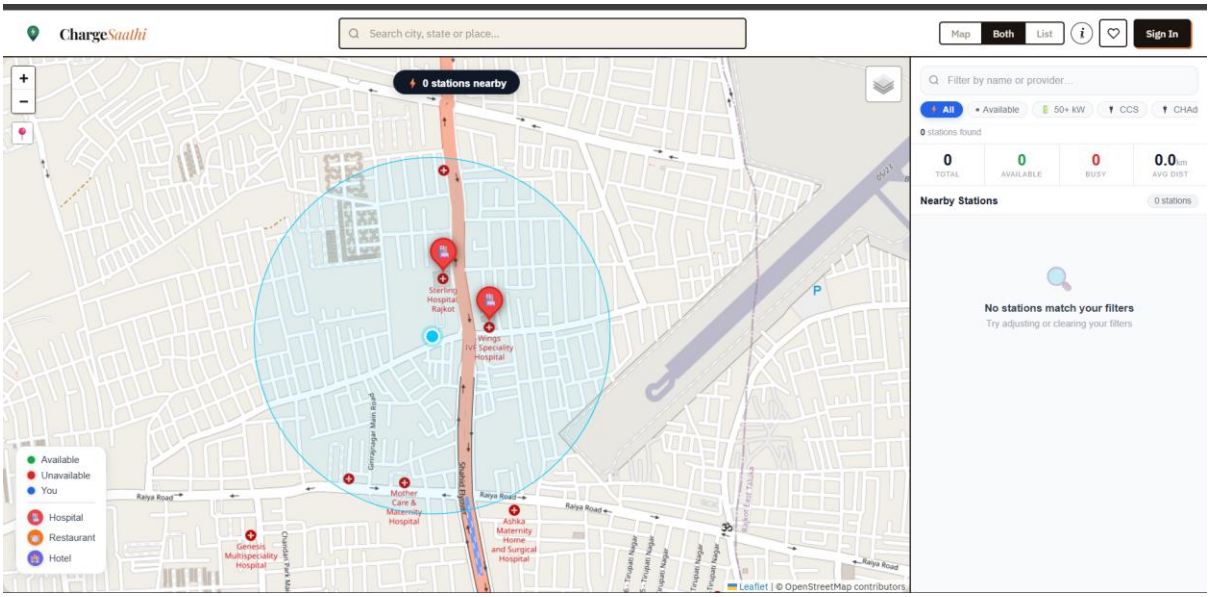
    No

Project Outcome Documents

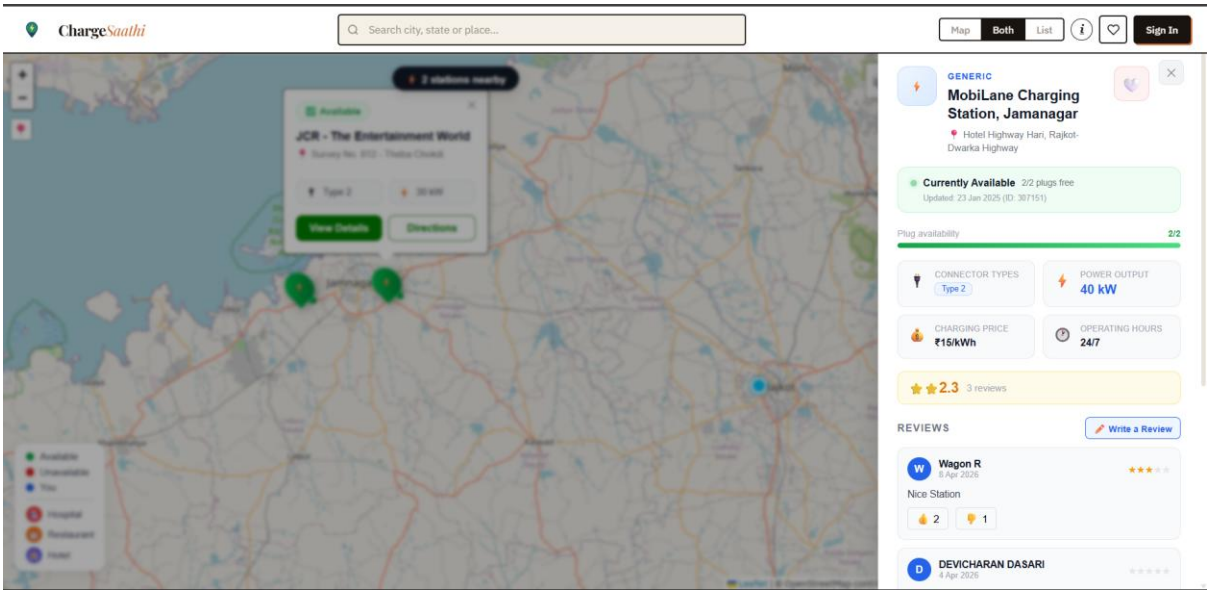
Screen 1: Home Page — Map + List View (Both Mode)



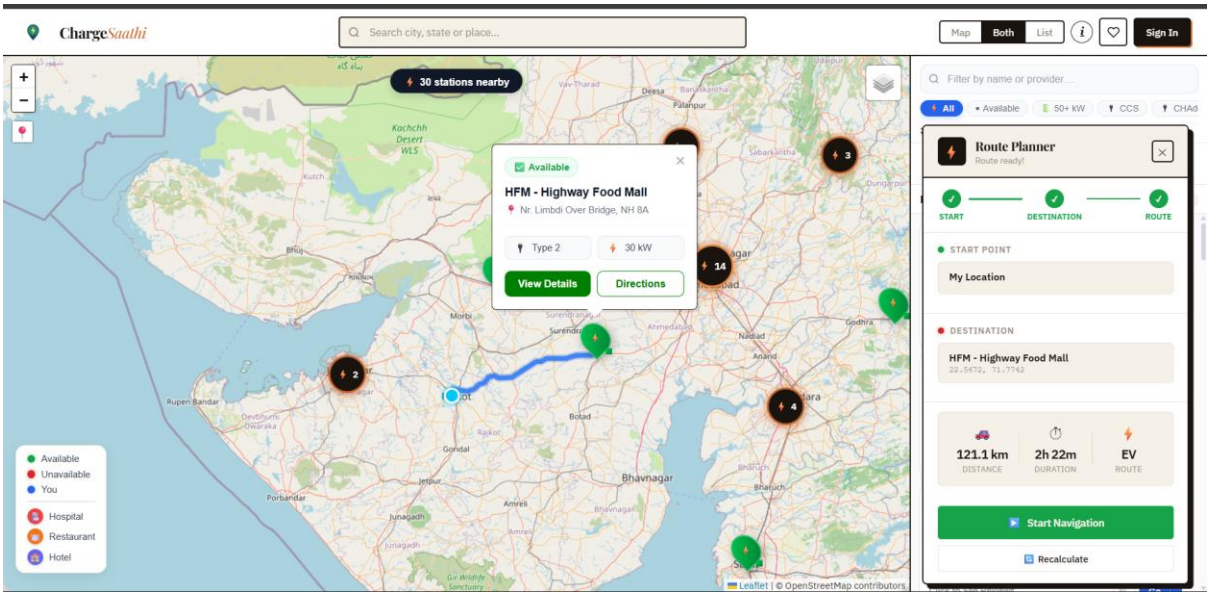
Screen 2: Map with POI Overlay (Zoom Level 15+)



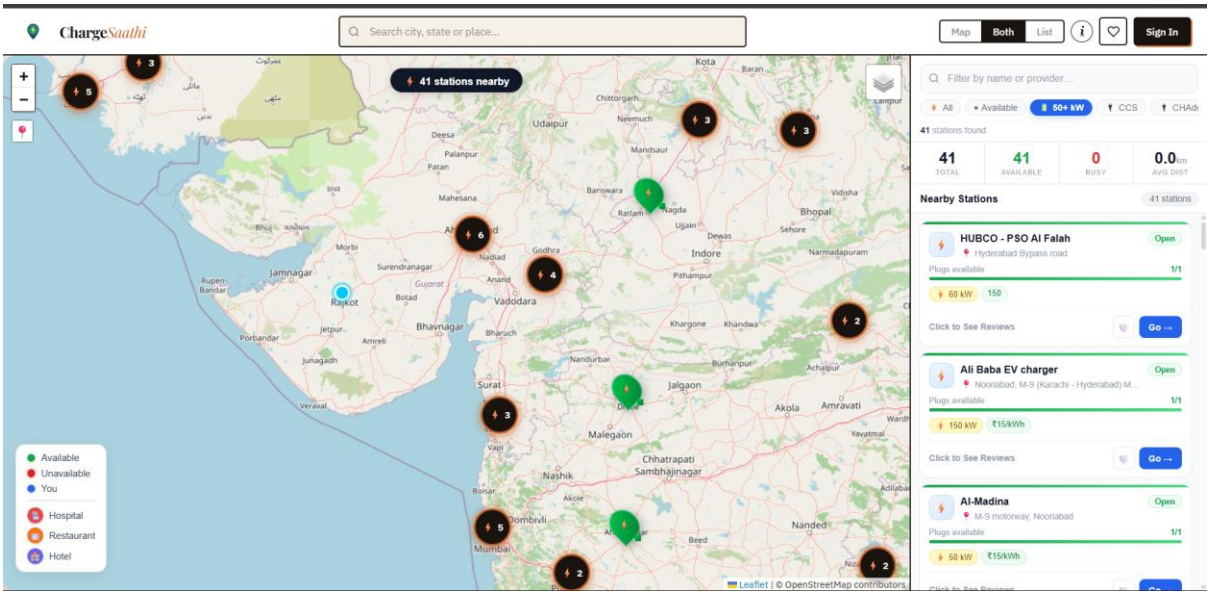
Screen 3: Station Details Modal



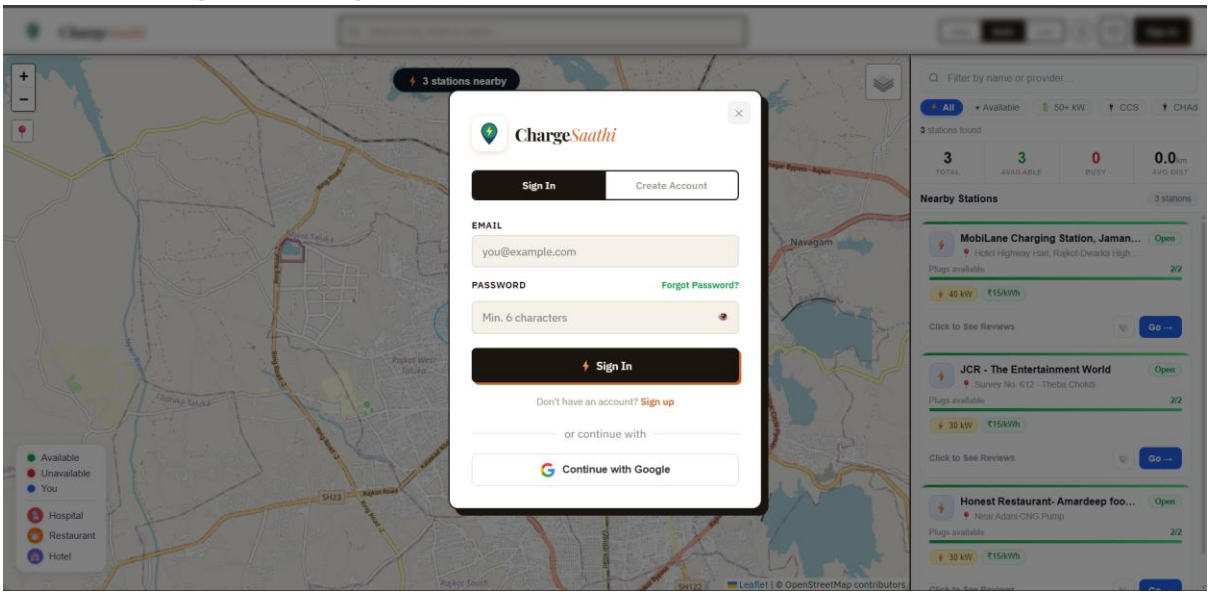
Screen 4: Station Details Modal



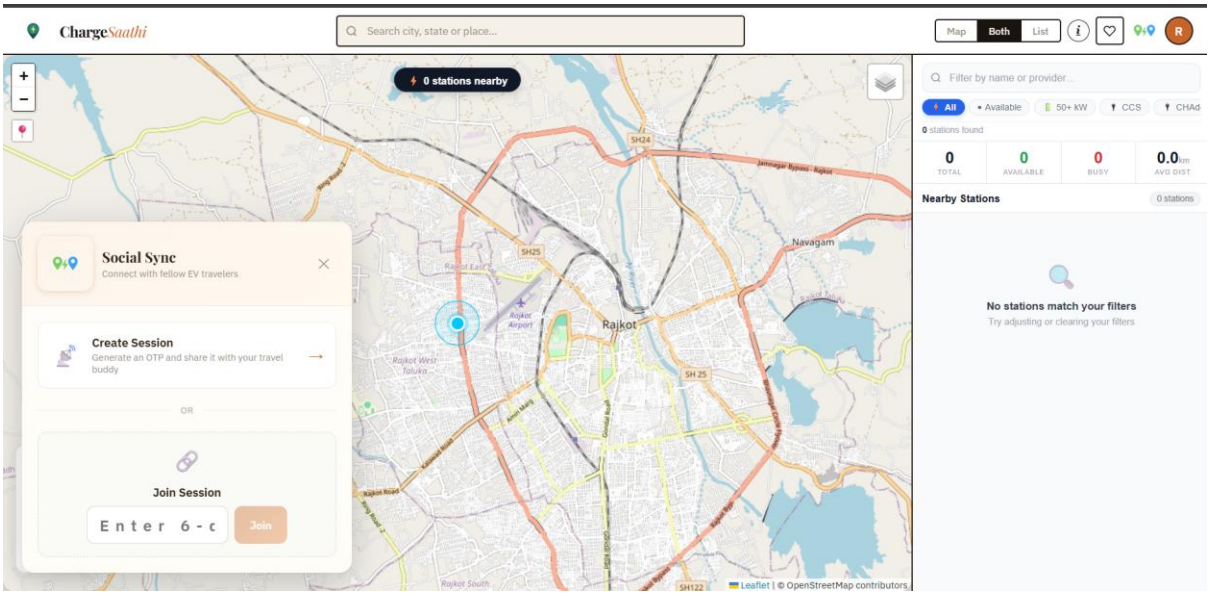
Screen 5: Search and Filter Panel



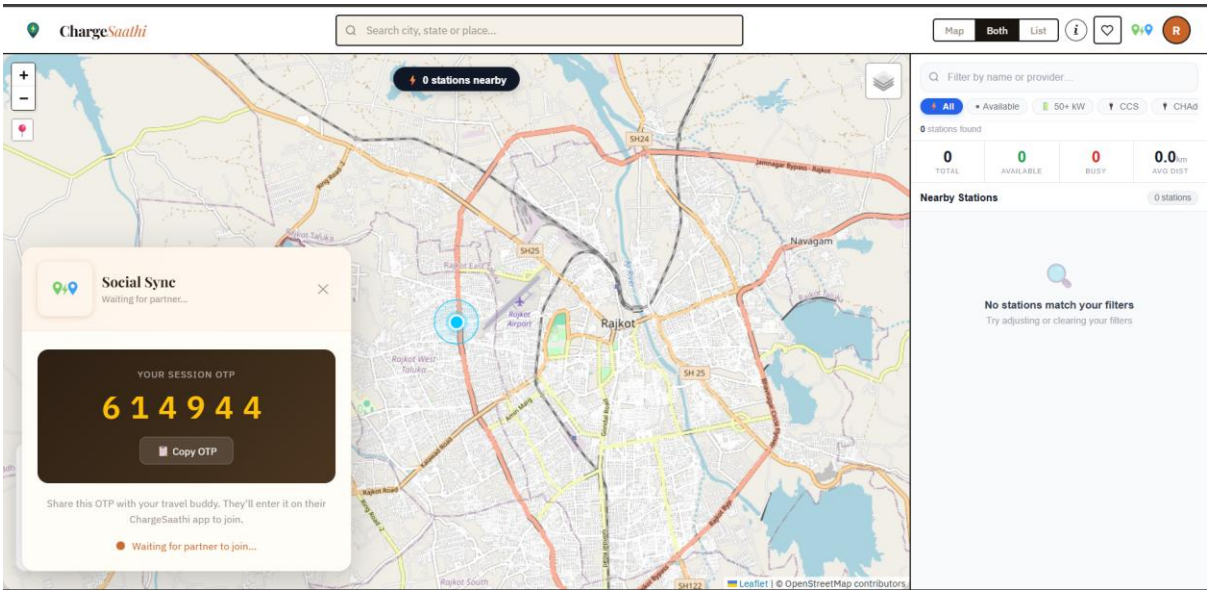
Screen 6: Sign In / Login



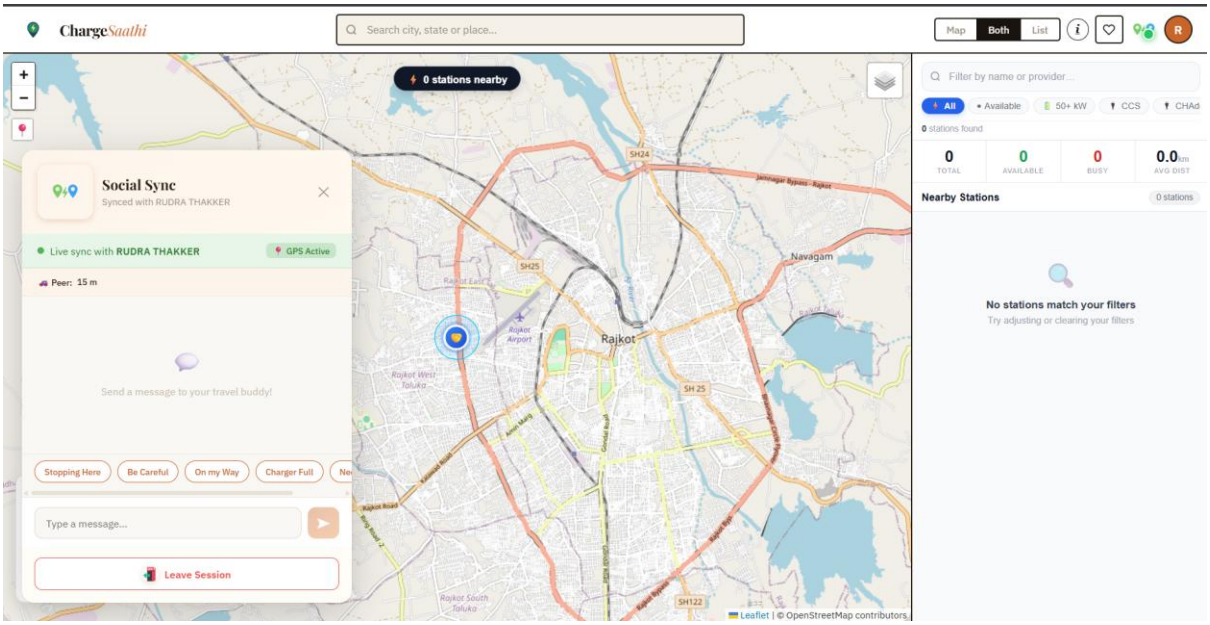
Screen 7: Social Sync



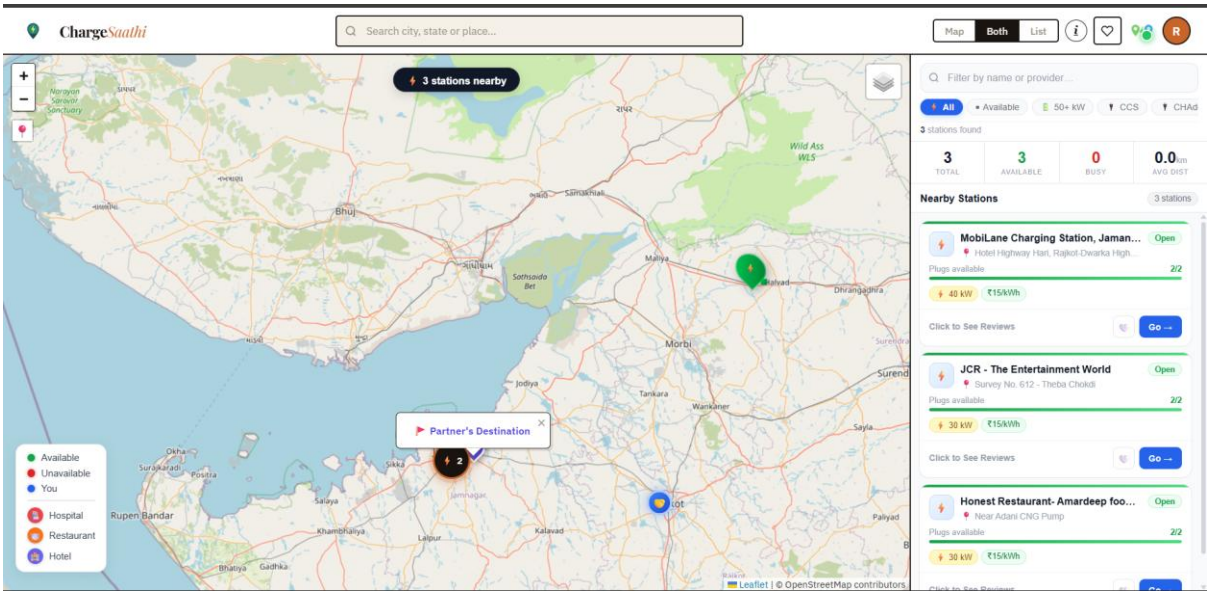
Screen 7: Social Sync



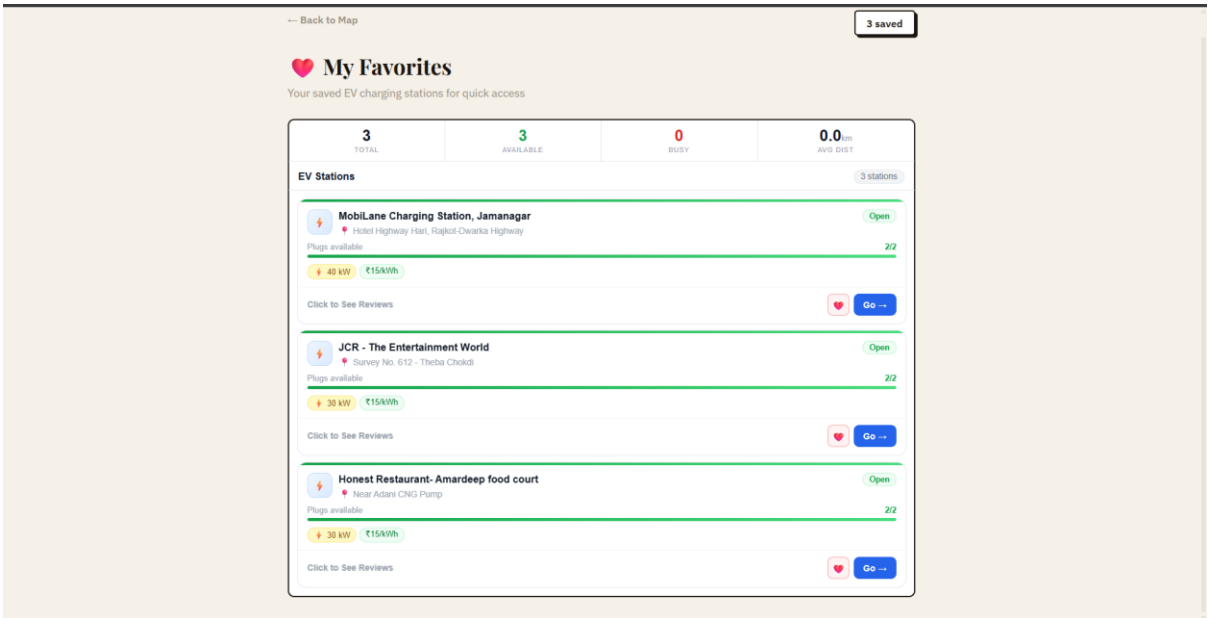
Screen 7: Social Sync



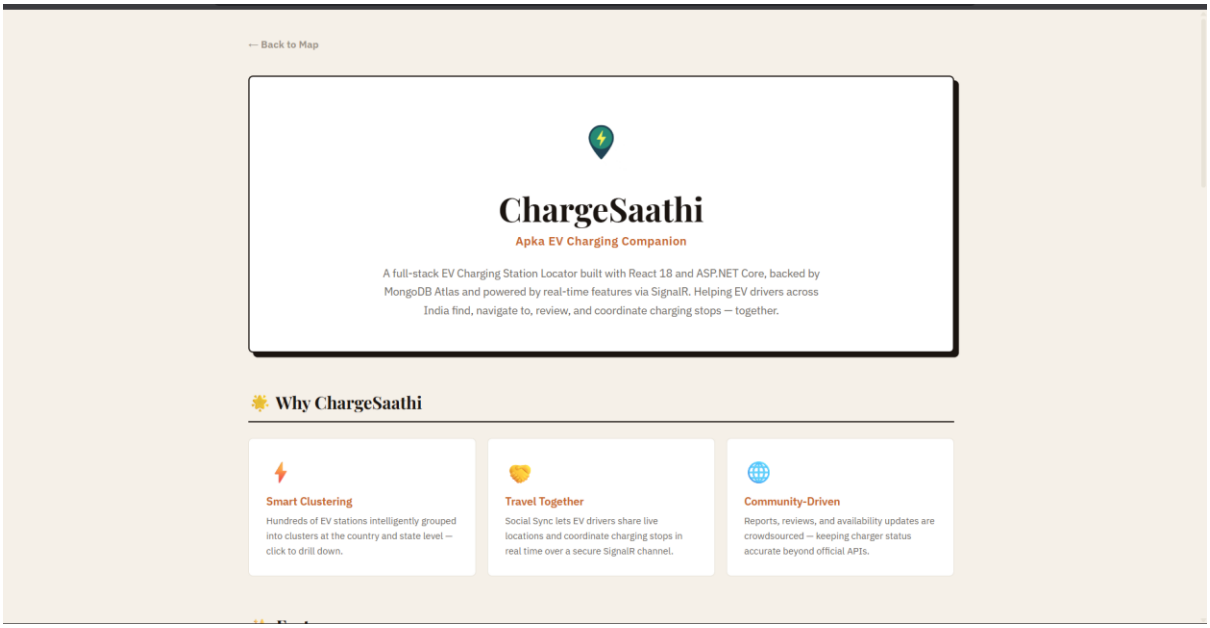
Screen 7: Social Sync



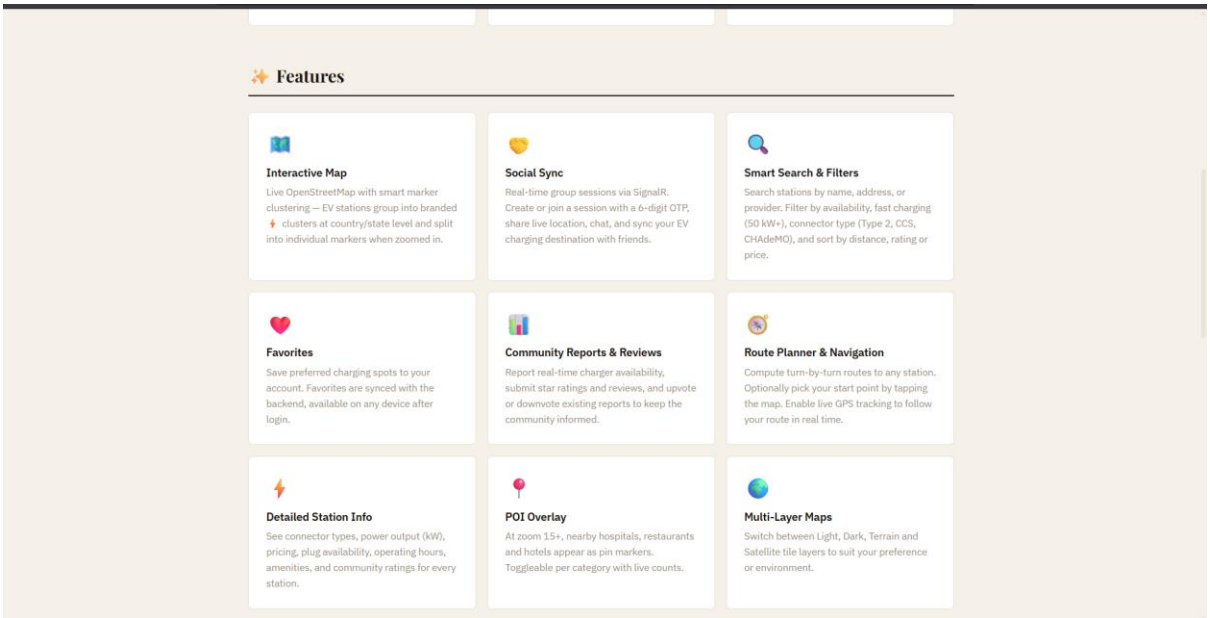
Screen 8: Favorites



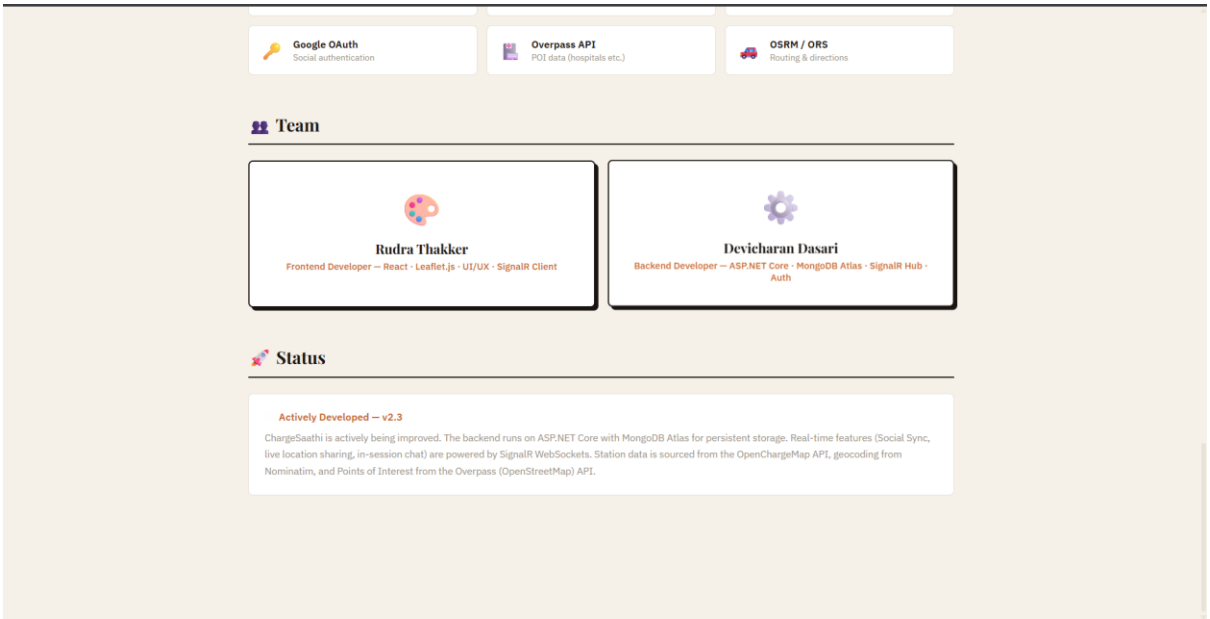
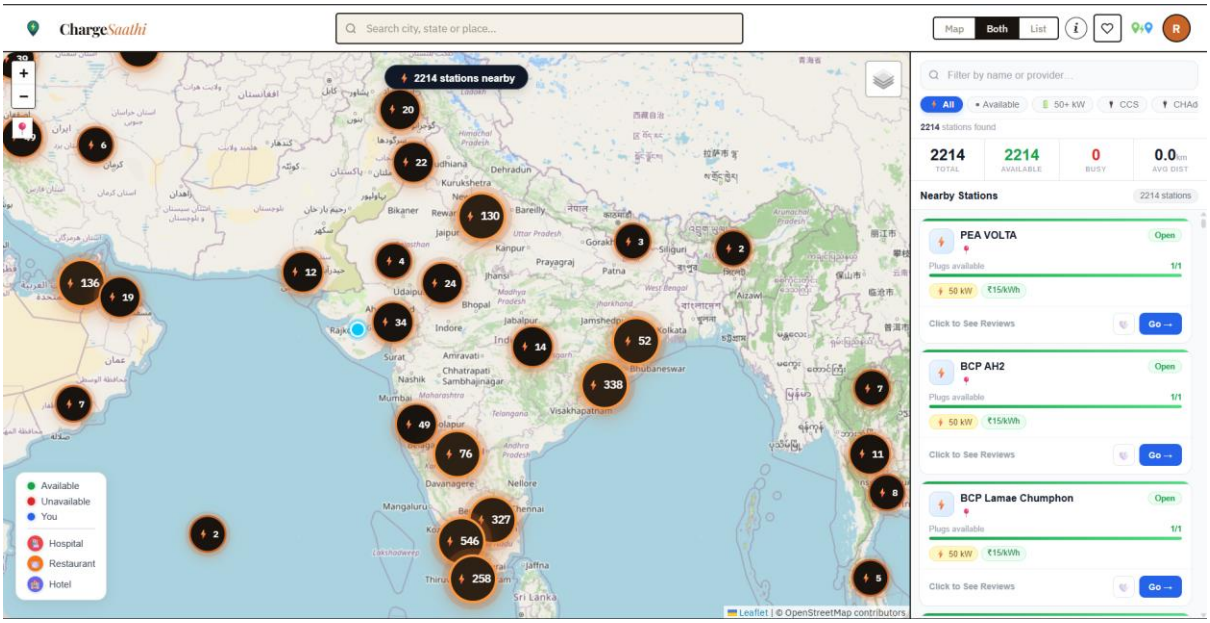
Screen 9: About Page



Screen 9: About Page



Screen 9: About Page



Screen 10: Zoom Out View